



УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ



**Развој open-source TinyMCE додатка за колаборативно
уређивање текста засновано на CRDT структури
података**

Мастер рад
Студијски програм: Рачунарство и информатика

Студент:

Вељко Вељовић, бр. инд. 1937

Ментор:

Проф. др Милош Богдановић

Ниш, октобар 2026. година

Универзитет у Нишу
Електронски факултет

**РАЗВОЈ OPEN-SOURCE TINYMCE ДОДАТКА ЗА КОЛАБОРАТИВНО
УРЕЂИВАЊЕ ТЕКСТА ЗАСНОВАНО НА CRDT СТРУКТУРИ ПОДАТАКА**

**DEVELOPMENT OF AN OPEN-SOURCE TINYMCE PLUGIN FOR
COLLABORATIVE TEXT EDITING BASED ON CRDT DATA STRUCTURES**

Мастер рад
Студијски програм: Рачунарство и информатика

Студент: Вељко Вељовић, бр. инд. 1937

Ментор: Проф. др Милош Богдановић

Задатак: Истражити особине Conflict-free Replicated Data Type (CRDT) структура података и потенцијал њихове употребе у системима за колаборативни рад. Истражити алгоритме за колаборативни рад засноване на употреби CRDT структура. У практичном делу имплементирати open-source додатак за TinyMCE едитор за колаборативно уређивање текста засновано на CRDT структури података, без потребе за централним закључавањем или трансформацијом операција.

Датум пријаве рада: 24.07.2026.

Датум предаје рада:

Датум одбране рада:

Комисија за оцену и одбрану:

1.

2.

3.

РАЗВОЈ OPEN-SOURCE TINYMCE ДОДАТКА ЗА КОЛАБОРАТИВНО УРЕЂИВАЊЕ ТЕКСТА ЗАСНОВАНО НА CRDT СТРУКТУРИ ПОДАТАКА

САЖЕТАК

Развој апликација намењених тимском раду наметнуо је потребу за системима који омогућавају да више корисника истовремено уређује исти документ, уз очување конзистентности података без нарушавања корисничког искуства. У раду су анализирана два основна приступа решавању овог проблема, Operational Transformation (OT) и Conflict-free Replicated Data Type (CRDT), при чему је посебна пажња посвећена особинама CRDT структура података, као што су конвергенција, коначна конзистентност, комутативност, асоцијативност и идемпотентност. Представљена је подела CRDT приступа на state-based и operation-based моделе, као и начин примене CRDT структура за колаборативно уређивање текста кроз операције уметања и брисања елемената. У практичном делу рада имплементиран је систем за колаборативно уређивање текста заснован на CRDT структури података. Систем се састоји од CrdtCore библиотеке, реализоване у C#/NET окружењу, која садржи логику CRDT механизма, CrdtServer апликације која омогућава комуникацију са клијентима путем SignalR-а и асинхрону размену операција између серверских компоненти путем RabbitMQ-а, као и TinyMCE додатка који повезује текстуални едитор са CRDT механизмом. Поред текста, CRDT приступ примењен је и на форматирање садржаја, чиме је омогућено колаборативно уређивање стилова и других атрибута текста. Реализован је и механизам за рад у офлајн режиму, који омогућава наставак рада у случају привременог губитка комуникације са сервером, као и накнадну синхронизацију локално насталих промена након успостављања везе. На овај начин систем омогућава континуиран рад корисника и у случају привремене недоступности серверске стране, уз очување конзистентности и конвергенције садржаја.

Кључне речи: CRDT, колаборативно уређивање текста, TinyMCE, офлајн режим, RabbitMQ, SignalR, форматирање текста, дистрибуирани системи.

DEVELOPMENT OF AN OPEN-SOURCE TINYMCE PLUGIN FOR COLLABORATIVE TEXT EDITING BASED ON CRDT DATA STRUCTURES

ABSTRACT

The development of applications designed for teamwork has created a need for systems that allow multiple users to edit the same document simultaneously while preserving data consistency without compromising the user experience. This thesis analyzes two main approaches to this problem, Operational Transformation (OT) and Conflict-free Replicated Data Type (CRDT), with particular attention given to the properties of CRDT data structures, such as convergence, eventual consistency, commutativity, associativity, and idempotence. The distinction between state-based and operation-based approaches is also presented, along with the use of CRDT structures for collaborative text editing through insertion and deletion operations. In the practical part of the thesis, a collaborative text editing system based on a CRDT data structure was implemented. The system consists of the CrdtCore library, developed in C#.NET and containing the CRDT logic, the CrdtServer application, which enables client communication via SignalR and asynchronous exchange of operations between server components using RabbitMQ, and a TinyMCE plugin that connects the text editor to the CRDT mechanism. In addition to text content, the CRDT approach is also applied to text formatting, enabling collaborative editing of styles and other text attributes. An offline mode mechanism was also implemented, allowing users to continue working in the event of a temporary loss of communication with the server and subsequently synchronize locally generated changes after the connection is restored. In this way, the system enables continuous user operation even when the server side is temporarily unavailable, while preserving the consistency and convergence of the content.

Keywords: CRDT, collaborative text editing, TinyMCE, offline mode, RabbitMQ, SignalR, text formatting, distributed systems.

СПИСАК КОРИШЋЕНИХ СКРАЋЕНИЦА

	СКРАЋЕНИЦЕ
API	A pplication P rogramming I nterface
CmRDT	C ommutative R eplicated D ata T ype
CRDT	C onflict-free R eplicated D ata T ype
CSS	C ascading S tyle S heets
CvRDT	C onvergent R eplicated D ata T ype
HTML	H yper T ext M arkup L anguage
HTTP	H ypertext T ransfer P rotocol
JSON	J ava S cript O bject N otation
LWW	L ast W rite W ins
OT	O perational T ransformation
RGA	R eplicated G rowable A rray
TCP	T ransmission C ontrol P rotocol
URL	U niform R esource L ocator
WAL	W rite- A head L og

Садржај

1. Увод.....	6
2. Репликација у дистрибуираним системима.....	8
2.1. Репликација са једним лидером.....	8
2.2. Синхрона и асинхрона репликација	10
2.3. Додавање нових реплика	11
2.4. Отказ чворова и failover механизам.....	12
2.5. Репликација са више лидера (multi-leader).....	13
2.6. Репликација без лидера (leaderless)	14
3. CRDT структуре података и Operational Transformation.....	15
3.1. Основни концепт CRDT структура	15
3.2. Особине CRDT структура.....	15
3.3. State-based CRDT	16
3.4. Operation-based CRDT.....	17
3.5. Operational Transformation	18
4. CRDT у колаборативном уређивању текста.....	20
4.1. Представљање текста CRDT структуром.....	20
4.2. Додавање и брисање елемента.....	21
4.3. Проблем испреплетаности елемената и Fugue алгоритам	22
4.4. Peritext модел за колаборативно форматирање текста.....	23
5. Практична реализација	25
5.1. TinyMCE едитор.....	25
5.2. Асинхрона комуникација путем RabbitMQ-а.....	25
5.3. WebSocket и SignalR комуникација.....	27
5.4. Архитектура система	29
5.5. Имплементација CRDT механизма	30
5.5.1. CrdtCore библиотека	30
5.5.2. CrdtServer.....	38
5.5.3. Клијентска страна и TinyMCE додатак	44
6. Закључак.....	55
7. Литература	56

1. Увод

Развој интернет технологија и пораст броја апликација намењених тимском раду довели су до потребе за системима који омогућавају да више корисника истовремено уређује исти документ. Алати попут Google Docs-а, Notion-а и сличних платформи показали су да корисници очекују могућност да истовремено, у реалном времену, врше измене над истим садржајем, без обзира на то да ли се налазе на истој локацији или су географски удаљени.

Овакав начин рада поставља специфичан технички проблем: када више корисника независно и конкурентно мења исти документ, неопходно је обезбедити да све измене буду сачуване и да сви учесници на крају виде исто, конзистентно стање документа. Наивна решења овог проблема, попут закључавања документа за време док га један корисник уређује или прихватања само последње извршене измене, значајно ограничавају могућност истовременог рада и нарушавају очекивано корисничко искуство. Због тога је потребно применити сложеније механизме који омогућавају да се конкурентне измене обједине без губитка података и без нарушавања намере корисника који је измену унео.

У пракси су се за решавање овог проблема развила два основна приступа, **Operational Transformation (OT)** и **Conflict-free Replicated Data Type (CRDT)**. OT приступ, коришћен у различитим системима за колаборативно уређивање, ослања се на механизме трансформације конкурентних операција како би њихова примена довела до конзистентног стања. CRDT приступ, са друге стране, конзистентност постиже кроз сам дизајн структуре података и правила за примену операција, чиме се омогућава да различите репликае документа након обраде истих измена конвергирају ка истом стању. Ова својства чине CRDT приступ погодним за системе који захтевају истовремени рад више корисника, као и за сценарије у којима није могуће обезбедити сталну мрежну повезаност.

Циљ овог мастер рада је истраживање CRDT структура података и њихове примене у системима за колаборативно уређивање текста, као и развој и имплементација система који омогућава истовремено уређивање текста од стране више корисника. Посебан фокус стављен је на пројектовање и имплементацију CRDT механизма за рад са текстуалним садржајем, обраду конкурентних измена и обезбеђивање конвергенције различитих реплика документа. Поред основних текстуалних операција, у раду је реализована и подршка за колаборативно уређивање форматирања текста, чиме је CRDT механизам проширен на стилове и друге атрибуте текстуалног садржаја.

У практичном делу мастер рада имплементиран је систем за колаборативно уређивање текста интегрисан са TinyMCE едитором. Систем омогућава размену и обраду измена између различитих инстанци едитора, без потребе за централним закључавањем или трансформацијом операција. Комуникација између клијентских инстанци и серверске апликације реализована је коришћењем SignalR-а, док је за асинхрону размену операција између серверских компоненти употребљен RabbitMQ. На овај начин омогућена је обрада конкурентних измена и њихова дистрибуција до повезаних инстанци система.

Поред рада у реалном времену, у систему је имплементиран и механизам за рад у офлајн режиму. Корисник може наставити са уређивањем документа и у случају привременог губитка комуникације са сервером, при чему се локално настале измене чувају и обрађују након поновног успостављања везе. Након успостављања комуникације, локалне измене се синхронизују са осталим репликама документа, чиме се омогућава наставак рада без

губитка претходно извршених измена. Овим је обезбеђена подршка за сценарије у којима долази до привремене недоступности мреже или серверске стране.

Рад је организован тако да су најпре представљени основни концепти дистрибуираних система и репликације, који пружају шири контекст за разумевање колаборативног уређивања. Након тога описане су CRDT структуре података, њихове основне особине и приступи њиховој имплементацији, као и OT приступ као алтернативно решење проблема конкурентних измена. У наставку су представљени CRDT приступи намењени колаборативном уређивању текста, као и начин представљања, уметања и брисања елемената и обраде конкурентних измена.

У практичном делу рада представљени су TinyMCE едитор, коришћене технологије за комуникацију и архитектура система. Описана је имплементација CRDT механизма која обухвата CrdtCore библиотеку, CrdtServer апликацију и клијентску страну са TinyMCE додатком. Посебно су описани механизми за синхронизацију измена путем SignalR-а и RabbitMQ-а, као и подршка за колаборативно уређивање форматираног текста и рад у офлајн режиму. Описани су и тестови којима је проверена конвергенција реплика, а на крају су изведени закључци рада.

2. Репликација у дистрибуираним системима

Репликација представља процес одржавања копија истих података на више међусобно повезаних рачунара путем мреже. Овај приступ омогућава дистрибуирано чување и обраду података, чиме се унапређују перформансе, доступност и поузданост система. Постоји више разлога за примену репликације у системима база података:

- **Смањење латенције приступа подацима** – Репликацијом се подаци могу позиционирати ближе крајњим корисницима (географски дистрибуирани системи), чиме се смањује време потребно за приступ подацима.
- **Повећање доступности и отпорности на грешке** – У случају отказа једног чвора, други чворови који садрже копију података могу наставити са радом, чиме се обезбеђује континуитет система и повећава његова поузданост.
- **Повећање перформанси кроз скалирање читања** – Више реплика може паралелно одговарати на упите који се односе на читање, чиме се растеређује примарни сервер и повећава укупни капацитет система за обраду захтева.

Међутим, одржавање више копија истих података уводи додатну сложеност, јер је неопходно обезбедити њихову међусобну усклађеност. Уколико се измена изврши на једној реплици, она се мора на одговарајући начин пренети и на остале реплике. У дистрибуираном систему измене не морају стићи до свих чворова истовремено, па реплике у одређеном тренутку могу садржати различите вредности. Ова појава је посебно значајна када више реплика може истовремено да прихвати измене над истим подацима.

Начин на који се измене пропадају између реплика утиче на време потребно за њихово усклађивање и на гаранције које систем пружа корисницима. Према начину на који се измене података пропадају између чворова, репликација се може поделити на **синхрону** и **асинхрону**. Код синхроне репликације измена се, пре потврде операције, прослеђује и другим репликама, док се код асинхроне репликације измена најпре прихвата на примарној реплици, а затим накнадно прослеђује осталим репликама.

Према начину организације чворова и управљању изменама података, разликују се модели репликације **са једним лидером (single-leader)**, **са више лидера (multi-leader)** и **без лидера (leaderless)**. Код single-leader модела један чвор има улогу примарног носиоца измена, док се остале реплике углавном користе за читање и преузимање података. Multi-leader модел омогућава да више чворова истовремено прихвата измене, што повећава могућност појаве конкурентних измена. Leaderless модел не подразумева један централни чвор који је одговоран за све измене, већ више реплика може независно прихватати операције.

Различити модели репликације пружају различите гаранције у погледу доступности, перформанси и конзистентности података. Због тога је приликом пројектовања дистрибуираног система неопходно размотрити на који начин ће се измене ширити између реплика и какво стање података корисник може очекивати приликом читања.

2.1. Репликација са једним лидером

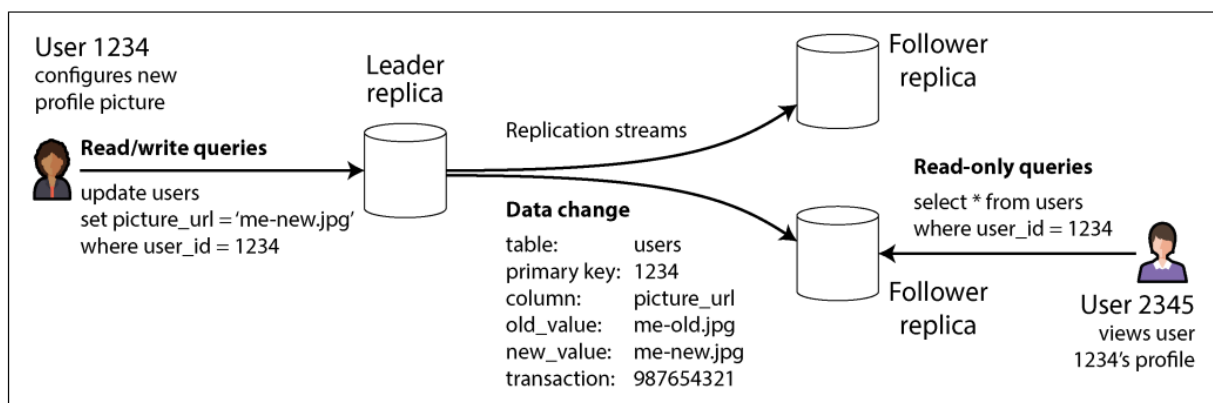
У системима са репликацијом, сваки чвор који садржи копију базе података назива се реплика. Како би се обезбедила усклађеност података између реплика, неопходно је да се измене извршене на једном чвору на одговарајући начин пренесу и примене на осталим чворовима у систему. Начин на који се ове измене прослеђују и обрађују зависи од модела

репликације.

Један од најчешће коришћених модела је репликација са једним лидером, позната и као primary-backup или active-passive модел. У овом приступу постоји један централни чвор, који се назива лидер (leader), док остали чворови имају улогу реплика (followers). Лидер је одговоран за прихватање и обраду операција измене података, док реплике одржавају копије података које добијају од лидера.

Рад оваквог модела репликације, приказан на слици 2.1, може се описати кроз следеће кораке:

- **Обрада уписа на лидеру** – Сви захтеви за измену података, као што су INSERT, UPDATE и DELETE, усмеравају се ка лидеру. Лидер извршава операцију над својом локалном копијом базе података и на тај начин постаје извор промена које треба пренети осталим репликама.
- **Бележење промена** – Након прихватања операције, информације о извршеној промени бележе се у репликационом логу. У зависности од конкретног система базе података, за ову сврху могу се користити различити облици логова, као што је **WAL (Write-Ahead Log)**. Лог садржи информације на основу којих реплике могу поновити промене извршене на лидеру.
- **Репликација промена на остале чворове** – Лидер прослеђује забележене промене осталим репликама. Реплике примају ове информације и примењују их над својим локалним копијама базе података. Уколико се измене примењују истим редоследом као на лидеру, реплике могу одржавати стање које одговара стању лидера.
- **Обрада операција читања** – Захтеви за читање могу се обрађивати на лидеру, али и на једној или више реплика. Коришћењем реплика за операције читања могуће је расподелити оптерећење система и повећати број захтева које систем може истовремено да обради. Међутим, код асинхроне репликације реплика може каснити за лидером, па читање са различитих чворова у истом тренутку не мора увек дати идентичан резултат.



Слика 2.1. – Обрада читања и уписа код репликације са једним лидером.

Једна од значајних предности овог модела је то што се операције измене обрађују на једном месту. На тај начин се поједностављује одређивање редоследа операција и избегава ситуација у којој више чворова истовремено покушава да измени исте податке. Лидер на тај начин представља јединствени извор измена, док реплике преузимају те измене и одржавају њихове копије.

Међутим, овакав приступ ствара зависност од лидера. Уколико лидер постане недоступан,

операције уписа не могу нормално да се наставе све док се не изабере нови лидер. Овај процес назива се избор лидера (leader election). Његова сврха је да се обезбеди наставак рада система и избегне ситуација у којој више чворова истовремено покушава да преузме улогу лидера.

2.2. Синхрона и асинхрона репликација

Један од најважнијих аспеката репликације јесте начин на који се измене пропадају између лидера и осталих реплика, односно да ли се репликација одвија синхронно или асинхронно. Избор начина репликације директно утиче на латенцију, доступност система, могућност губитка података и степен конзистентности између реплика.

Код синхроне репликације, лидер приликом обраде измене чека потврду од једне или више реплика да је измена успешно примљена и, у зависности од конфигурације система, примењена, пре него што операцију сматра успешно завршеном и одговори клијенту. На овај начин се смањује могућност да се након отказа лидера изгубе измене које су претходно потврђене клијенту, јер су оне већ пренете на синхроне репликае. Међутим, овакав приступ повећава латенцију операција уписа, јер њихов завршетак зависи и од доступности и одзива синхроних реплика. Уколико реплика постане недоступна или значајно касни са обрадом измене, упис може бити успорен или привремено блокиран.

Са друге стране, код асинхроне репликације, лидер не чека потврду осталих реплика пре него што одговори клијенту. Након што се измена успешно примени на лидеру, операција се сматра завршеном, док се промена накнадно прослеђује осталим репликама. На овај начин се постиже мања латенција и већа пропусност система, јер брзина обраде уписа не зависи директно од тренутне доступности осталих реплика. Недостатак овог приступа је могућност постојања репликационог застоја (replication lag), при чему репликае одређени временски период могу имати старије стање у односу на лидера. Поред тога, у случају отказа лидера пре него што су све измене прослеђене репликама, постоји могућност губитка последњих потврђених измена.

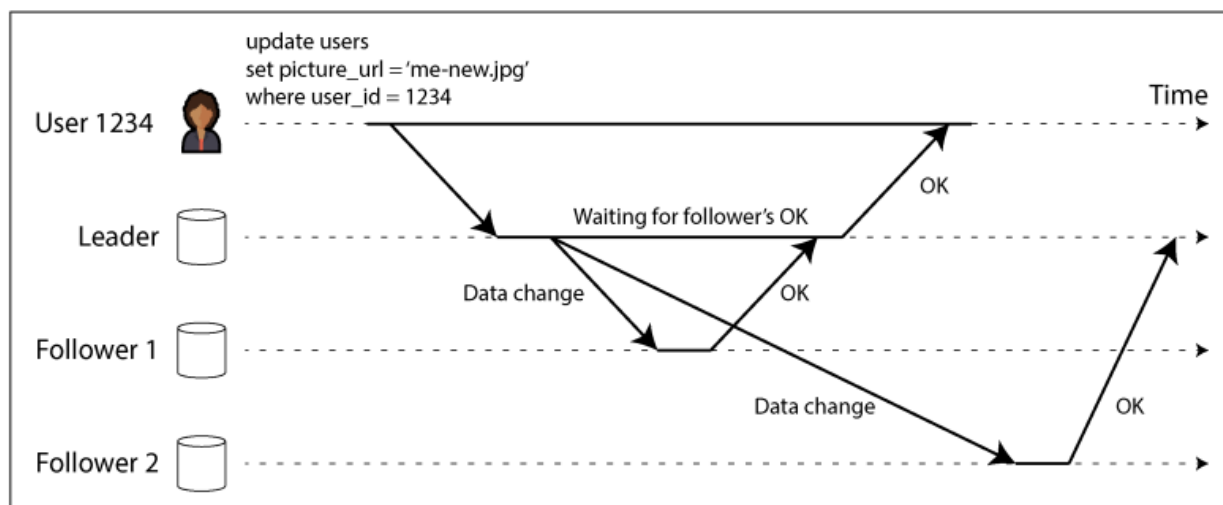
У пракси се често користе и комбинације синхроне и асинхроне репликације. На пример, једна реплика може бити конфигурирана као синхрона, док се остале репликае ажурирају асинхронно. На овај начин систем може обезбедити да бар једна реплика добије измену пре него што се упис потврди клијенту, док се остале репликае могу ажурирати без утицаја на латенцију операције.

Важно је напоменути да синхрона и асинхрона репликација не представљају потпуно одвојене моделе организације реплика, већ начин на који се измене преносе и потврђују између њих. На пример, single-leader систем може користити и синхрону и асинхронну репликацију, у зависности од захтева система.

Избор између синхроне и асинхроне репликације представља компромис између конзистентности, доступности и перформанси. Синхрона репликација пружа јаче гаранције у погледу доступности потврђених измена на више реплика, али уз већу латенцију и зависност од њихове доступности. Асинхрона репликација омогућава бржу обраду захтева и већу отпорност на привремену недоступност реплика, али доводи до могућности привремено различитих стања и губитка последњих измена у случају отказа лидера.

На слици 2.2. приказан је пример single-leader репликације у којем је једна реплика

конфигурисана као синхрона, док се друга ажурира асинхронно. Код синхроне реплике лидер чека потврду успешне репликације пре завршетка операције, док се измене ка асинхронној реплици прослеђују без чекања на њену потврду.



Слика 2.2. – Пример single-leader репликације са синхроним и асинхроним ажурирањем реплика.

2.3. Додавање нових реплика

У системима са репликацијом често постоји потреба за додавањем нових реплика, било ради повећања доступности, скалирања система или замене неисправних чворова. Кључно питање у овом процесу јесте како обезбедити да нова реплика поседује конзистентну и тачну копију података са лидера. Једноставно копирање датотека базе података са једног чвора на други није адекватно решење, јер се подаци у бази непрекидно мењају услед активних трансакција. Такав приступ може довести до неконзистентног стања података. Закључавање базе током копирања решило би овај проблем, али би истовремено нарушило доступност система, што није прихватљиво у системима који захтевају високу доступност.

Због тога се у пракси користи поступак који комбинује иницијални снимак стања базе и накнадну репликацију промена. Процес иницијализације нове реплике може се описати кроз следеће кораке:

1. **Креирање конзистентног снимка (snapshot)** – Креира се тренутни снимак стања базе података на лидеру, по могућству без закључавања целокупне базе. Овај механизам је често подржан од стране система базе података и може се користити и за потребе резервног копирања.
2. **Пренос снимка на нову реплику** – Креирани снимак се преноси на нови чвор који ће преузети улогу реплике и користи се за формирање њеног почетног стања.
3. **Синхронизација промена путем репликационог лога** – Након преузимања снимка, нова реплика се повезује са лидером и преузима све измене које су настале након тренутка креирања снимка. Ове измене се обично преносе коришћењем репликационог лога.
4. **Достизање конзистентног стања** – Када реплика обради све измене које су настале након креирања снимка, достиже стање које одговара актуелном стању лидера и наставља да прима нове промене.

Овакав приступ омогућава додавање нових реплика без потребе за дуготрајним

заустављањем рада система. Комбинација иницијалног снимка и репликационог лога омогућава да се нова реплика постепено доведе у актуелно стање, након чега може равноправно учествовати у систему репликације.

2.4. Отказ чворова и failover механизам

У дистрибуираним системима откази чворова су неизбежни и могу настати услед хардверских грешака, мрежних проблема, софтверских грешака или планираног одржавања. Циљ система са високом доступношћу јесте да обезбеди континуитет рада упркос оваквим догађајима, уз минималан утицај на кориснике. У системима који користе single-leader репликацију могу се разликовати два основна сценарија: отказ реплике и отказ лидера.

Када реплика престане са радом, на пример услед пада система или прекида мрежне комуникације, њен опоравак је релативно једноставан. Реплика чува информације о променама које је примила од лидера и може утврдити до које тачке је обрадила репликациони лог. Након поновног покретања, реплика се поново повезује са лидером, преузима измене које су настале током њене недоступности и примењује их како би достигла актуелно стање лидера. Након успешног опоравка, реплика наставља нормално да прима нове промене. Уколико је реплика дуже време била недоступна или је у систему постојао велики број уписа, процес њеног сустизања може захтевати значајне ресурсе и додатно оптеретити и лидера и саму реплику.

Отказ лидера представља сложенији проблем, јер је неопходно пренети његову улогу на један од доступних чворова. Овај процес се назива **failover**. У оквиру failover процеса једна од реплика се бира за новог лидера, клијенти се усмеравају ка новом лидеру, а остале реплике се реконфигуришу тако да промене преузимају од новог лидера. Failover може бити покренут ручно од стране администратора или аутоматски.

Аутоматски failover се обично састоји од следећих корака:

1. **Детекција отказа лидера** – Систем користи механизме као што су временска ограничења (timeout) и провере доступности како би утврдио да лидер више није доступан.
2. **Избор новог лидера** – Једна од доступних реплика бира се за новог лидера на основу дефинисаних критеријума. Један од важних критеријума може бити ажурност података, односно количина промена које је реплика већ примила. У зависности од архитектуре система, избор новог лидера може укључивати и механизме консензуса.
3. **Реконфигурација система** – Клијенти се усмеравају ка новом лидеру, док се остале реплике повезују са новим лидером и настављају да преузимају нове промене. Уколико се стари лидер поново појави, неопходно је спречити да настави са прихватањем уписа као лидер и правилно га укључити у систем као реплику.

Failover у системима репликације представља сложен процес који може довести до губитка последњих промена, неконзистентности или привременог прекида рада система уколико није правилно имплементиран. Посебан изазов представљају мрежни проблеми, јер је у дистрибуираном систему могуће да чвор буде функционалан, али привремено недоступан другим чворовима. Због тога систем мора пажљиво разликовати стварни отказ чвора од привремене мрежне недоступности. Циљ failover механизма је да се обезбеди наставак рада система уз очување што већег степена конзистентности и минимизовање

губитка података.

2.5. Репликација са више лидера (multi-leader)

Један од главних недостатака single-leader репликације јесте чињеница да сви захтеви за упис морају проћи кроз једног лидера. Уколико тај чвор постане недоступан, на пример услед мрежног прекида, операције уписа не могу нормално да се обрађују све док се не обезбеди други доступан лидер. Као алтернатива овом приступу развијен је модел репликације са више лидера (multi-leader репликација), у којем више чворова може истовремено прихватати операције уписа. Сваки од тих чворова има улогу лидера за клијенте који му приступају, али истовремено представља и реплику осталих лидера.

У овом моделу, измене података настале на једном лидеру пропадају се ка осталим лидерима. Као и код single-leader репликације, пропагација измена може бити синхрона или асинхрона. У пракси се чешће користи асинхрони приступ, јер омогућава да сваки лидер настави са прихватањем уписа чак и када је комуникација са другим лидерима привремено прекинута. На овај начин се повећава доступност система и омогућава независнији рад различитих чворова. Са друге стране, асинхрона репликација може довести до тога да различити лидери у одређеном тренутку поседују различите верзије података.

Главни изазов multi-leader репликације представља **појава конфликта при конкурентним уписима**. До конфликта долази када различити лидери независно и истовремено измене исте податке. Пошто се измене између лидера најчешће пропадају асинхронно, није могуће гарантовати да ће сви лидери у тренутку обраде измене имати исту верзију података. Због тога систем мора да обезбеди механизме за детекцију и решавање насталих конфликта.

За решавање конфликта у multi-leader репликацији користе се различите технике, у зависности од захтева апликације и природе података. Један од једноставнијих приступа је **Last Write Wins (LWW)**, код којег се задржава вредност повезана са последњом изменом према утврђеном критеријуму редоследа, док се конкурентна измена одбацује. Иако је овај приступ једноставан за имплементацију, може довести до трајног губитка једне од конкурентних измена.

Други приступ подразумева **ручно решавање конфликта**, при чему се конкурентне верзије података чувају и препуштају апликацији или кориснику да одреди коначни резултат. Овај приступ је сличан решавању конфликта приликом спајања различитих верзија датотека у системима за контролу верзија. Савременији системи могу користити и аутоматизоване механизме за решавање конфликта, при чему се правила за њихово разрешавање дефинишу у складу са природом података и захтевима апликације.

Multi-leader репликација је посебно погодна за системе у којима је неопходно да више географски удаљених чворова може независно прихватати операције уписа. На пример, корисници из различитих региона могу приступати локалном лидеру, док се измене накнадно пропадају ка осталим лидерима. На тај начин се смањује латенција приступа и повећава доступност система, али се истовремено повећава сложеност синхронизације и решавања конфликта.

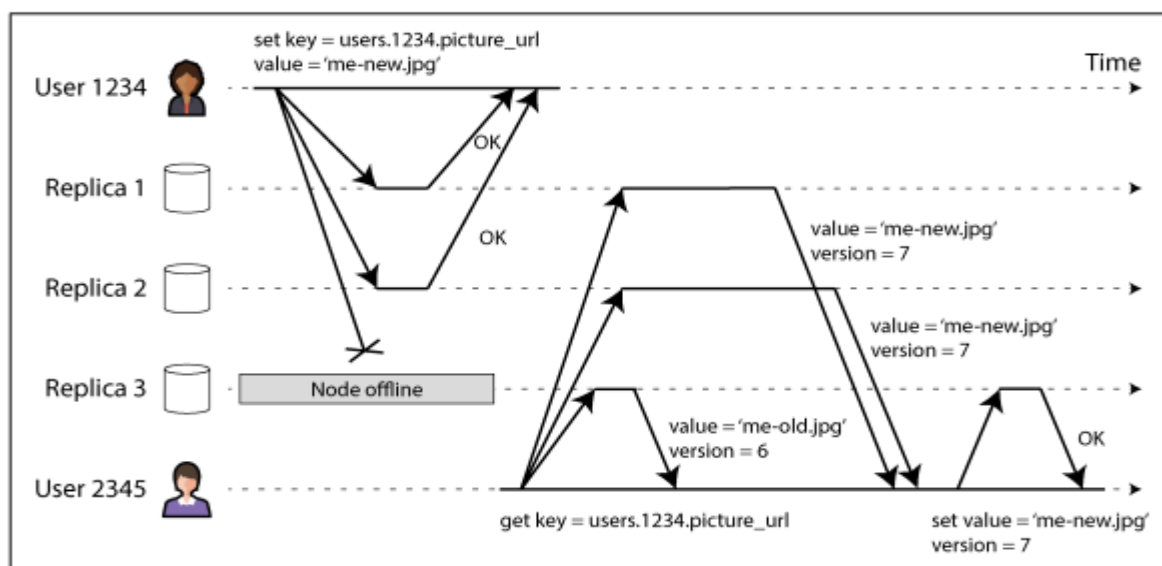
2.6. Репликација без лидера (leaderless)

Репликација без лидера (leaderless репликација) представља приступ у којем се напушта концепт централног лидера, па више чворова може директно прихватати захтеве за упис података. За разлику од single-leader и multi-leader модела, не постоји један чвор који има посебну улогу у одређивању редоследа свих операција. Уместо тога, подаци се реплицирају на више чворова који независно обрађују захтеве.

Приликом уписа, клијент шаље податке на више реплика, најчешће паралелно. Операција се може сматрати успешно завршеном након што одређени број реплика потврди пријем података. Овај приступ омогућава систему да настави са радом чак и када део реплика привремено није доступан. Конкретан број реплика које морају потврдити операцију зависи од конфигурације система и жељеног нивоа доступности и конзистентности.

Када се недоступна реплика поново прикључи систему, она може имати застарело стање јер није примила измене настале током периода своје недоступности. Због тога је неопходно обезбедити механизме помоћу којих реплика може да надокнади пропуштене измене. Један од приступа је **read repair**, код којег се приликом читања уочава разлика између верзија података на различитим репликама и застареле реплика се ажурирају најновијом верзијом. Други приступ је коришћење механизма за накнадну синхронизацију реплика, којим се недостајуће измене преносе на реплику након њеног повратка у систем.

Читање података у leaderless моделу такође може подразумевати приступ већем броју реплика. Добијене вредности се затим упоређују на основу информација о њиховој верзији, као што су верзиони бројеви или временски жигови. Уколико различите репликае садрже различите верзије истог податка, систем примењује дефинисана правила како би одредио која вредност представља актуелно стање. На слици 2.3. приказан је пример у којем реплика привремено није доступна, након чега се њено стање накнадно усаглашава са осталим репликама.



Слика 2.3. – Пример асинхроне репликације са привремено недоступном репликом и накнадним усаглашавањем њеног стања.

3. CRDT структуре података и Operational Transformation

У овом поглављу биће представљен концепт CRDT структура података и њихова примена у дистрибуираним системима. Биће описане основне карактеристике CRDT структура, различити приступи њиховој имплементацији, као и механизми који омогућавају конвергенцију реплика без потребе за централним управљањем конфликтима. На крају поглавља описан је и Operational Transformation (OT), као алтернативни приступ обради конкурентних измена.

3.1. Основни концепт CRDT структура

CRDT је структура података намењена за употребу у дистрибуираним системима, код којих се више реплика истих података може мењати независно и конкурентно. Ове структуре података гарантују да ће све реплике конвергирати ка истом конзистентном стању, чак и када се ажурирања примењују различитим редоследом или када поједини чворови имају привремене прекиде у раду.

Једна од основних карактеристика CRDT приступа је коначна конзистентност, односно својство да ће све реплике временом достићи конзистентно стање уколико све на крају обаве исти скуп операција. При томе, измене могу бити извршене независно и без непосредне комуникације између свих учесника.

CRDT структуре омогућавају да се конкурентне измене обраде без потребе за ручним решавањем конфликта или применом сложених алгоритама за њихову детекцију и обраду. Подаци се могу реплицирати на више чворова, при чему свака реплика може независно прихватити измене и размењивати их са осталим репликама. Одговарајућим механизмима спајања измена обезбеђује се да реплике на крају достигну исто стање.

CRDT приступ се може применити на различите типове структура података, као што су бројачи, скупови и графови, али и на сложеније структуре попут JSON објеката. За сваки тип структуре дефинисане су операције које омогућавају конкурентно ажурирање података уз очување конзистентности реплика.

У зависности од начина представљања и размене измена, CRDT структуре се могу поделити на **state-based** и **operation-based** CRDT структуре. Иако се ова два приступа разликују у начину синхронизације реплика, њихов заједнички циљ је обезбеђивање конвергенције и коректног комбиновања конкурентних измена.

CRDT структуре су посебно погодне за системе за колаборативно уређивање текста, јер омогућавају да више корисника истовремено врши измене над локалним репликама документа, након чега се те измене могу синхронизовати и објединити у јединствено стање документа. У наставку ће бити детаљније описане особине CRDT структура и различити приступи њиховој имплементацији.

3.2. Особине CRDT структура

CRDT структуре података поседују низ особина које омогућавају њихову примену у дистрибуираним системима у којима се подаци могу мењати на више реплика истовремено. Ове особине омогућавају да се конкурентне измене обрађују без потребе за централним управљањем и да различите реплике документа након синхронизације

достигну исто стање.

Једна од најзначајнијих особина CRDT структура је **конвергенција**. Она подразумева да ће све реплике које су обрадиле исти скуп измена на крају достићи идентично стање, без обзира на редослед којим су измене примљене. Ово својство је од посебног значаја у дистрибуираним системима јер мрежно кашњење, привремени прекиди комуникације и различити редоследи пристизања порука могу довести до тога да реплике у одређеном тренутку имају различита стања. CRDT механизми обезбеђују да се такве привремене разлике могу успешно превазићи након размене свих насталих измена.

Са конвергенцијом је повезано својство **коначне конзистентности**. Оно подразумева да реплике не морају у сваком тренутку имати идентично стање, већ да ће, под условом да се нове измене више не појављују и да све настале измене на крају буду достављене свим репликама, систем временом достићи конзистентно стање. На овај начин могуће је дозволити привремене разлике између реплика без нарушавања коначне конзистентности система.

За постизање конвергенције важна су одређена алгебарска својства операција и механизма за спајање стања. Једно од њих је **комутативност**, која подразумева да резултат примене две операције не зависи од њиховог редоследа. У контексту дистрибуираних система, ово омогућава да две реплике које приме исте операције различитим редоследом могу достићи исто коначно стање.

Значајно својство је и **асоцијативност**. Она омогућава да се више измена комбинује у различитим груписањима без промене коначног резултата. Ово је посебно значајно приликом спајања измена које су настале на већем броју реплика, јер систем не мора да зависи од начина на који су појединачне измене груписане приликом синхронизације.

CRDT структуре такође могу обезбедити **идемпотентност** операција спајања. Идемпотентност подразумева да поновљена примена исте измене не мења већ постигнуто стање. Ова особина је значајна у дистрибуираним системима јер поруке могу бити достављене више пута, па њихова поновна обрада не би требало да произведе додатне или неочекиване измене. На тај начин се повећава отпорност система на поновљено достављање података.

3.3. State-based CRDT

CRDT структуре се, према начину на који се измене примењују између реплика, могу поделити на две основне групе, state-based CRDT и operation-based CRDT структуре. Основна разлика између ова два приступа огледа се у информацијама које се размењују приликом синхронизације. Док се код state-based приступа између реплика размењује њихово стање, код operation-based приступа размењују се операције које представљају измене извршене над подацима.

State-based CRDT, познат и као CvRDT, представља приступ у коме се синхронизација реплика заснива на размени тренутних стања. Свака реплика поседује сопствено локално стање над којим може независно да врши измене. Након тога, стање једне реплике може бити прослеђено другој реплици, која га комбинује са сопственим стањем.

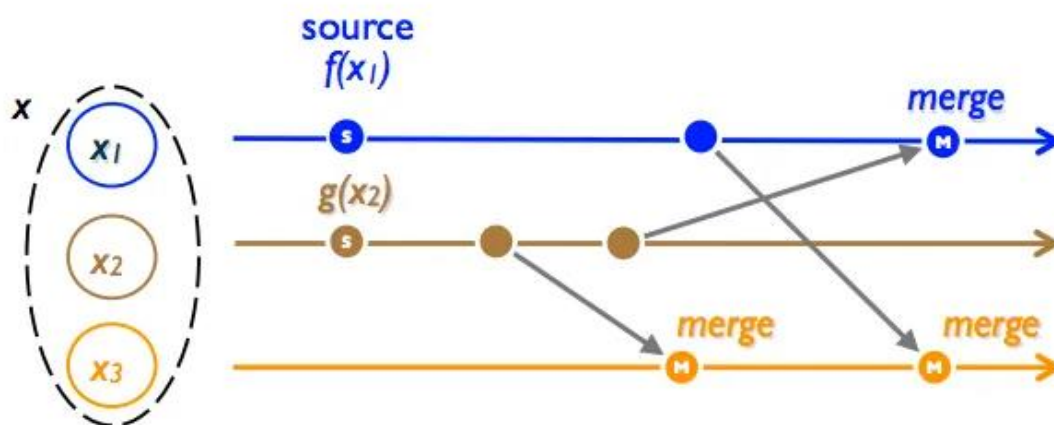
Основни механизам state-based CRDT приступа представља merge функција. Њена улога је да на основу два стања произведе ново стање које садржи информације из оба улазна

стања. На овај начин се измене настале на различитим репликама могу објединити без потребе за одређивањем јединственог редоследа свих претходно извршених операција. Merge функција такође треба да буде комутативна, асоцијативна и идемпотентна.

Једна од значајнијих предности state-based CRDT приступа је његова једноставност са становишта синхронизације. Реплика не мора да води рачуна о томе које су конкретне измене довеле до тренутног стања друге реплике. Довољно је да добије њено стање и примени дефинисану функцију за спајање. Ово омогућава да се пропуштене измене надокнаде приликом наредне синхронизације.

Са друге стране, главни недостатак овог приступа представља количина података која се може размењивати током синхронизације. Уколико је структура података велика, пренос целокупног стања може представљати значајно мрежно и меморијско оптерећење. Овај проблем је нарочито изражен код система у којима се подаци често мењају, а појединачне измене су мале у односу на укупну величину структуре.

Иако state-based приступ пружа једноставан и робустан механизам синхронизације, количина података која се размењује може ограничити његову примену код великих структура. Због тога се у одређеним системима користи operation-based приступ, код којег се уместо целокупног стања размењују операције које представљају појединачне измене. Размена стања између три реплике приказана је на слици 3.1.



Слика 3.1. – State-based CRDT, где три реплике размењују своја стања.

3.4. Operation-based CRDT

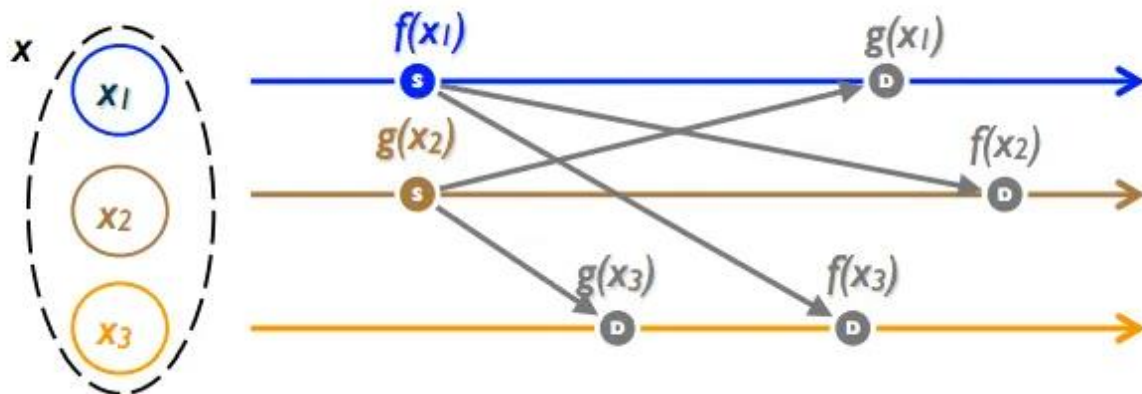
Operation-based CRDT, познат и као CmRDT, представља приступ синхронизацији реплика код којег се између реплика размењују операције које представљају измене над подацима, уместо целокупног стања структуре. Свака реплика може независно да извршава локалне операције, након чега се настале операције прослеђују другим репликама како би их и оне примениле над својим локалним стањем.

Основна идеја operation-based приступа је да се промена података представи као операција која садржи све информације неопходне за њену примену на удаљеној реплици. На пример, операција може описивати додавање, уклањање или измену одређеног елемента. Након што друга реплика прими такву операцију, она је примењује над сопственим стањем. На овај начин се између реплика не мора преносити целокупна структура

података, већ само информације о насталим изменама.

Неопходно је обезбедити да операције буду правилно достављене, идентификоване и примењене, као и да њихова примена не доведе до различитих стања реплика. Уколико операција зависи од друге операције или од одређеног стања структуре, потребно је обезбедити и одговарајућу обраду таквих зависности.

Основна предност operation-based приступа је ефикаснија размена измена, јер се између реплика преносе само операције које описују настале промене, уместо целокупног стања структуре. Са друге стране, state-based приступ обично пружа једноставнији механизам синхронизације, јер се реплике могу усклађивати разменом њихових стања и применом јединствене merge функције, без потребе за праћењем и појединачном обрадом свих претходно извршених операција. Размена операција између реплика приказана је на слици 3.2.



Слика 3.2. – Operation-based CRDT, операције извршене над једном репликом се прослеђују другим репликама, које их затим примењују над својим локалним копијама података.

3.5. Operational Transformation

ОТ представља још један приступ решавању проблема конкурентних измена у системима за колаборативно уређивање. Основна идеја ОТ приступа је да се операције које су настале независно на различитим репликама трансформишу пре њихове примене, тако да њихово коначно дејство буде међусобно усклађено. На овај начин више корисника може истовремено вршити измене над истим документом, при чему се измене обрађују тако да се очува конзистентност реплика.

Оба приступа имају исти основни циљ, а то је омогућавање конзистентног колаборативног рада више корисника, али се разликују у начину на који обрађују конкурентне измене. За разлику од CRDT приступа, који конвергенцију постиже одговарајућим дизајном структуре података и правила за комбиновање измена, ОТ се ослања на трансформацију операција. Када реплика прими операцију која је настала на основу стања документа које се разликује од њеног тренутног стања, операција се трансформише у односу на локалне операције које су у међувремену извршене. Тиме се операција прилагођава тренутном стању документа пре њене примене.

Типичан пример представља истовремено уметање текста на исту позицију. Ако два

корисника независно уметну различите карактере на истој позицији, директна примена обе операције различитим редоследом може довести до различитих резултата. ОТ приступ трансформише другу операцију у односу на прву, чиме се њена позиција прилагођава новонасталом стању документа. На тај начин обе измене могу бити сачуване, а реплике могу достићи исто коначно стање.

У класичним имплементацијама ОТ приступа често се користи централизовани сервер који има улогу координатора између клијената. Клијенти шаљу своје операције серверу, који их прима, одређује њихов редослед и, по потреби, врши њихову трансформацију пре него што их проследи осталим клијентима. Оваква архитектура поједностављује координацију конкурентних измена, али истовремено ствара зависност од централне компоненте. Недоступност сервера може спречити размену и обраду измена између клијената.

Један од главних изазова ОТ приступа представља сложеност алгоритама за трансформацију операција. За различите комбинације операција потребно је дефинисати одговарајућа правила трансформације, као што су истовремено уметање, брисање и комбинације уметања и брисања. Са повећањем броја типова операција повећава се и број случајева које је потребно обрадити, што може отежати имплементацију и тестирање система.

4. CRDT у колаборативном уређивању текста

CRDT структуре података представљају погодан приступ за имплементацију система за колаборативно уређивање текста, јер омогућавају да више корисника истовремено врши измене над различитим репликама истог документа. Свака реплика може независно да обрађује локалне измене, док се настале промене размењују између учесника и примењују над њиховим локалним структурама. Да би се обезбедила конзистентност и конвергенција између реплика, неопходно је дефинисати начин представљања елемената, као и правила за обраду уметања, брисања и конкурентних измена.

4.1. Представљање текста CRDT структуром

За разлику од традиционалног начина представљања текста, у којем се карактери најчешће посматрају као елементи низа којима се приступа на основу њихове позиције, CRDT приступ представља текст као колекцију елемената којима су додељени јединствени идентификатори и који имају дефинисан међусобни редослед. Овакво представљање је неопходно јер позиција елемената у тексту није стабилна у условима конкурентних измена.

На пример, ако једна реплика садржи текст „ABC“, позиција карактера „C“ је 2. Уколико друга реплика истовремено уметне карактер „X“ између „B“ и „C“, позиција карактера „C“ се мења. Због тога се операција која се ослања искључиво на нумеричку позицију може применити на погрешан елемент када се изврши над репликом која је у међувремену промењена.

CRDT приступ решава овај проблем тако што се елементи идентификују независно од њихове тренутне позиције у тексту. Сваки елемент поседује јединствени идентификатор који омогућава да се конкретан елемент препозна на свим репликама. Поред идентификатора структура може чувати и информацију о односу елемента према другим елементима, на пример референцу на претходни елемент. На тај начин се позиција елемента не одређује само бројем у низу, већ се њихов положај може реконструисати на основу структуре и правила за одређивање редоследа.

Идентификатор елемента мора бити јединствен у оквиру целокупног документа како би се избегло да различите реплике креирају елементе са истим идентификатором. Један од начина за постизање јединствености је комбиновање идентификатора реплике са локалним бројачем операција или елемента. Реплика поседује сопствени идентификатор, док бројач представља редни број новокреираног елемента на тој реплици. Идентификатор се на тај начин може представити као пар идентификатора реплике на којој је елемент креиран и локалног бројача који се увећава приликом креирања нових елемената. Комбиновањем ова два податка обезбеђује се јединственост идентификатора на нивоу целокупне CRDT структуре, јер елементи креирани на различитим репликама имају различите вредности за идентификатор реплике, чак и у случају када имају исту вредност бројача.

Идентификатор елемента није важан само за његово јединствено препознавање, већ и за обраду конкурентних операција. Када две реплике истовремено уметну елементе на исту позицију, њихови идентификатори могу се користити као део детерминистичког правила за одређивање редоследа. На тај начин све реплике могу применити исто правило и добити исти распоред елемената, без обзира на редослед којим су примиле операције.

Поред сопственог идентификатора, елемент може садржати и информације о елементима који се налазе непосредно испред и иза њега у структури текста. На тај начин сваки елемент може бити повезан са својим суседима, што омогућава очување редоследа елемената и лакше одређивање њиховог положаја приликом обраде измена.

Овакво представљање омогућава да се положај елемента одређује на основу његових односа са осталим елементима, а не искључиво на основу тренутне позиције у тексту. Ово је посебно значајно код конкурентних измена, јер се позиције елемената могу мењати услед уметања и брисања, док њихови идентификатори и међусобни односи остају основа за реконструкцију структуре текста.

4.2. Додавање и брисање елемента

Операција уметања представља једну од основних операција над текстом у CRDT структури. Њена сврха је додавање новог елемента у постојећу структуру, при чему је неопходно одредити његов положај у односу на већ постојеће елементе. За разлику од традиционалног приступа, где се место уметања најчешће задаје нумеричком позицијом, CRDT приступ се ослања на односе између елемената и њихове јединствене идентификаторе.

Приликом локалног уметања новог елемента, реплика креира елемент са јединственим идентификатором и одређује његов однос према постојећим елементима. На овај начин место елемента није одређено само његовом тренутном позицијом, већ односом са осталим елементима. То омогућава да се операција уметања правилно обради и на другим репликама, чак и ако су оне у међувремену извршиле додатне измене.

Посебан случај настаје када више реплика истовремено умеће елементе на исто место. На пример, ако структура садржи текст „ABC“ и две реплике независно уметну елементе X и Y између B и C, након синхронизације обе реплике морају имати исти редослед елемената. Реплике ће након локалних измена имати различита стања.

$$A \rightarrow B \rightarrow X \rightarrow Y \rightarrow C$$
$$A \rightarrow B \rightarrow Y \rightarrow X \rightarrow C$$

Оба резултата могу бити исправна, под условом да све реплике применом истог правила добију исти резултат. Кључно је да редослед не зависи од тога која је операција прва стигла до одређене реплике. CRDT структура мора да дефинише детерминистичко правило на основу којег ће се одредити међусобни редослед конкурентно уметнутих елемената. Исто правило мора бити примењено на свим репликама, без обзира на редослед којим су операције примљене. На пример, ако је дефинисано да реплика са мањим идентификатором има предност у односу на реплику са већим идентификатором, обе реплике ће након пријема обе операције добити исти редослед.

$$A \rightarrow B \rightarrow X \rightarrow Y \rightarrow C$$

Детерминистичко одређивање редоследа представља основу за конвергенцију текстуалних реплика. Уколико све реплике приме исти скуп елемената и примене иста правила за њихово уређивање, њихове структуре ће након синхронизације имати исти редослед.

Брисање представља другу основну операцију над елементима текста. За разлику од уметања, код којег се у структуру додаје нови елемент, приликом брисања је потребно означити постојећи елемент као обрисан. У дистрибуираном окружењу физичко уклањање елемента из структуре може довести до проблема приликом обраде конкурентних операција, јер друге реплике могу и даље поседовати референце на тај елемент.

Због тога CRDT структуре за текст често користе приступ познат као **tombstone**. Код овог приступа обрисани елемент се не уклања физички из структуре, већ се означава као обрисан. Приликом приказивања текста такви елементи се занемарују, али остају присутни у унутрашњој структури података.

Описани приступ, у којем се елементи идентификују јединственим идентификаторима, повезују са осталим елементима структуре и на основу дефинисаних правила одређује њихов редослед, користи и RGA структура. RGA је CRDT структура намењена за рад са уређеним секвенцама, као што је текст, и омогућава да се елементи независно додају и бришу на различитим репликама уз очување конзистентности њиховог редоследа.

4.3. Проблем испреплетаности елемената и Fugue алгоритам

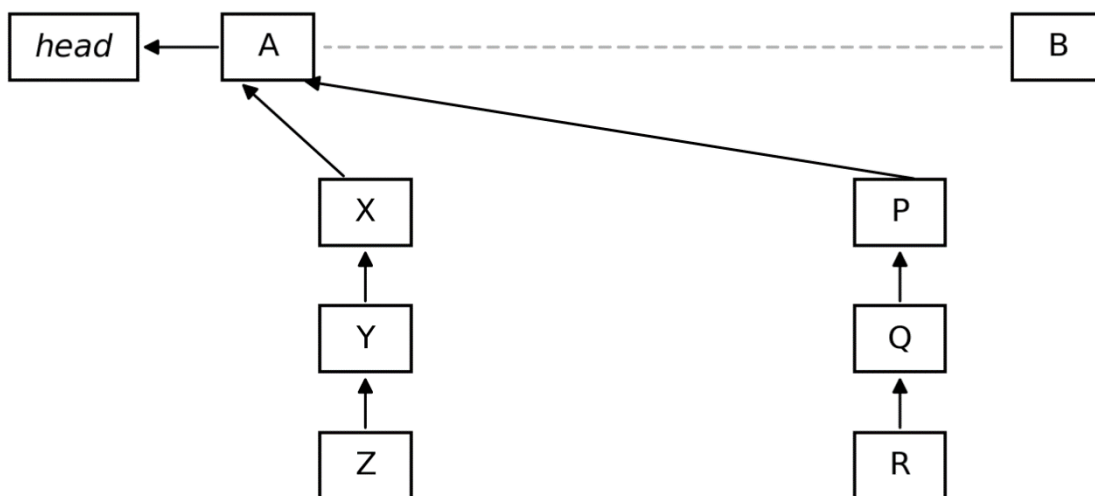
Правило за одређивање редоследа конкурентно уметнутих елемената, описано у претходном одељку, обезбеђује да све реплике постигну исти, детерминистички распоред елемената. Међутим, у литератури је показано да оваква, наизглед довољна правила, могу произвести нежељен ефекат приликом уметања дужих низова карактера, познат као проблем испреплетаности.

Проблем настаје када две или више реплика истовремено уметну по више узастопних карактера на исту позицију у тексту. На пример, ако полазна реплика садржи текст „AB“, а две реплике конкурентно уметну речи „XYZ“ и „PQR“ између елемената A и B, примена једноставног критеријума за одређивање редоследа, попут поређења идентификатора реплике, може довести до тога да коначан распоред елемената буде облика „XPYQZR“, односно да се карактери обе унете речи међусобно измешају уместо да свака реч остане очувана као целовит, нераздвојив низ. Овакав исход је формално коректан у смислу да ће све реплике добити идентичан резултат, чиме је формално задовољено својство конвергенције, али је семантички неисправан, јер корисник који је унео реч „XYZ“ очекује да она остане читљива као целина, а не измешана са туђим уносом.

Као решење овог проблема представљен је CRDT алгоритам под називом **Fugue**. За разлику од једноставног поређења идентификатора реплика, Fugue елементе организује у структуру налик стаблу, у којој сваки елемент чува референцу на елемент у односу на који је уметнут у тренутку свог настанка. При одређивању редоследа конкурентно уметнутих елемената, Fugue узима у обзир њихове односе са референтним елементима и начин на који су елементи уметнути. На тај начин се омогућава да узастопно унети карактери у оквиру исте операције у највећој мери остану груписани, уместо да буду раздвојени елементима насталим конкурентним операцијама.

Fugue притом задржава све особине CRDT структуре описане у претходним поглављима, односно конвергенцију, комутативност, асоцијативност и идемпотентност, и не захтева измену основног модела комуникације између реплика. Разлика у односу на класичне приступе попут RGA огледа се искључиво у правилу за одређивање релативног редоследа конкурентно уметнутих елемената, чиме је Fugue директно применљив као замена или унапређење постојећих секвенцијалних CRDT структура. Структура стабла за пример

конкурентног уметања низова „XYZ“ и „PQR“ приказана је на слици 4.1.



Слика 4.1. – Fugue стабло за пример конкурентног уметања низова „XYZ“ и „PQR“, где оба подстабла настају у односу на исти референтни елемент A.

4.4. Peritext модел за колаборативно форматирање текста

CRDT структуре података омогућавају да се конкурентне измене над текстом обједине тако да све реплике система након размене операција достигну исто стање. Међутим, подршка за обичан текст представља само један део проблема колаборативног уређивања. Савремени уређивачи текста омогућавају и примену различитих облика форматирања, као што су подебљавање (**bold**), курзив (*italic*), избор фонта и промена величине слова. Увођењем оваквих могућности, потребно је дефинисати и начин на који се форматирање понаша у присуству конкурентних измена.

Један од једноставнијих приступа за представљање форматирања јесте придруживање атрибута појединачним карактерима текста. На пример, сваком карактеру могу се придружити својства која одређују његов изглед. Иако је овај приступ релативно једноставан за имплементацију, конкурентне измене могу довести до неочекиваног преношења или губитка форматирања. Проблем постаје израженији када један корисник истовремено мења текст, док други примењује или уклања форматирање над истим делом документа.

Као решење овог проблема предложен је Peritext, модел намењен представљању и обради форматираног текста у дистрибуираним системима. Основна идеја Peritext модела јесте раздвајање садржаја текста од информација о његовом форматирању. Уместо да се форматирање директно чува у самим карактерима, оно се представља помоћу посебних интервала, односно *formatting span*-ова, који одређују део текста на који се одређено форматирање примењује.

Границе ових интервала не зависе искључиво од тренутне позиције карактера у тексту, већ се ослањају на стабилне идентификаторе елемената CRDT структуре. Ово је значајно јер

се позиција одређеног карактера може променити као последица конкурентног уметања или брисања текста. Стабилни идентификатори омогућавају да се границе форматирања повежу са конкретним елементима структуре, независно од њихове тренутне позиције у приказаном тексту.

Посебна пажња у Peritext моделу посвећена је очувању намере корисника. Код конкурентних измена није довољно само обезбедити да све реплике на крају садрже исто стање. Потребно је, у мери у којој то правила алгорита дозвољавају, очувати значење операција које су корисници извршили. На пример, ако један корисник проширује текст унутар форматираног дела, а други истовремено врши измену садржаја или граница тог форматирања, начин разрешавања конфликта треба да буде детерминистички и да одражава семантику извршених операција.

Peritext је првенствено усмерен на **inline форматирање текста**, односно на промене које се примењују на део текста, као што су подебљавање, курзив, врста фонта и величина слова. Структурни елементи документа који обухватају целе блокове, као што су наслови, листе, цитати или табеле, представљају другачији проблем и нису у потпуности обухваћени основним моделом. Код колаборативног уређивања, овакво форматирање подразумева неколико специфичних случајева које је потребно правилно обрадити како би се очувало очекивано понашање текста.

- **Истовремено форматирање и уметање** – Уколико се текст уметне унутар већ форматираног дела, новоуметнути текст треба да наследи одговарајуће форматирање. На тај начин се задржава очекивано понашање приликом истовременог уређивања текста и његовог форматирања.
- **Преклапање форматирања** – Различита форматирања која се примењују на исти део текста могу истовремено да постоје и да се комбинују. На пример, исти део текста може истовремено бити подебљан и приказан у курзиву.
- **Конфликтна форматирања** – До конфликта долази када корисници на исти део текста примене међусобно искључива форматирања, као што су различите боје текста. Пошто оба форматирања не могу истовремено да буду приказана, потребно је дефинисати детерминистичко правило на основу којег ће све реплике документа доћи до истог резултата.
- **Понашање на границама интервала** – Посебно је потребно дефинисати понашање форматирања приликом уметања текста непосредно пре или после постојећег форматираног дела. У зависности од врсте форматирања, нови текст може наследити форматирање суседног дела или остати ван постојећег интервала.

Peritext надограђује постојећу CRDT структуру за обичан текст додавањем операција које представљају промене форматирања. Основне операције над текстом су операције уметања и брисања карактера, док се промене форматирања представљају посебним операцијама које дефинишу почетак, крај и врсту форматирања. Приликом уклањања форматирања претходне операције се не мењају нити бришу, већ се генерише нова операција којом се мења њихов ефекат. На овај начин све операције остају део CRDT структуре и могу се доследно обрадити на свим репликама.

5. Практична реализација

У практичном делу рада имплементиран је систем за колаборативно уређивање текста заснован на CRDT структури података. За уређивање текста коришћен је TinyMCE, који је проширен сопственим додатком. Серверска страна система реализована је коришћењем C# и .NET технологија, док је за комуникацију између клијената и сервера коришћен SignalR. Комуникација између различитих серверских инстанци реализована је асинхроно, коришћењем RabbitMQ система за размену порука.

5.1. TinyMCE едитор

TinyMCE је open-source JavaScript едитор за уређивање текста (rich-text editor) који се може интегрисати са веб апликацијама. Корисницима омогућава креирање и измену текстуалног садржаја кроз интерфејс сличан класичним текстуалним едиторима, уз подршку за различите операције форматирања и уређивања садржаја. Едитор се може прилагодити потребама конкретне апликације кроз конфигурацију и систем додатака (plugins).

Једна од значајних карактеристика TinyMCE едитора је могућност његовог проширивања путем механизма додатака. Додатак представља независну компоненту која може додати нове функционалности едитору или изменити његово постојеће понашање. На овај начин могуће је приступити садржају едитора, реаговати на догађаје који настају током уређивања и програмски извршавати измене над документом.

За потребе овог рада коришћен је механизам додатака како би се између корисничких измена у TinyMCE едитору и CRDT структуре података успоставила веза. Када корисник изврши измену над текстом, додатак препознаје насталу промену и претвара је у одговарајућу CRDT операцију. Таква операција се затим може обрадити локално и проследити осталим репликама документа. Са друге стране, када се од удаљене реплике прими CRDT операција, додатак је обрађује и резултат измене примењује над садржајем TinyMCE едитора.

Архитектура TinyMCE-а омогућава да се ова два нивоа раздвоје. TinyMCE се користи за приказ и интеракцију са корисником, док додатак служи као посредни слој између едитора и механизма за колаборативно уређивање. На тај начин је могуће задржати стандардне могућности едитора, док се функционалност колаборативног уређивања додаје без измене саме TinyMCE библиотеке.

5.2. Асинхрона комуникација путем RabbitMQ-а

У имплементираном систему RabbitMQ се користи за асинхрону комуникацију између серверских инстанци и размену CRDT операција. Потреба за оваквом комуникацијом произилази из чињенице да више клијената може бити повезано на различите сервере, при чему сваки сервер одржава локалну реплику документа. Када клијент изврши измену, сервер на који је клијент повезан генерише одговарајућу CRDT операцију и прослеђује је RabbitMQ систему за размену порука. Остале серверске инстанце које учествују у раду над истим документом могу преузети операцију и применити је над својим локалним стањем. На тај начин се омогућава размена операција између серверских реплика.

RabbitMQ је систем за посредовање у размени порука (**message broker**) који омогућава

апликацијама и сервисима да међусобно размењују поруке без потребе за директном комуникацијом. Пошиљалац поруке не мора да познаје конкретну серверску инстанцу која ће је обрадити, већ поруку прослеђује RabbitMQ систему, који је затим усмерава одговарајућим примаоцима.

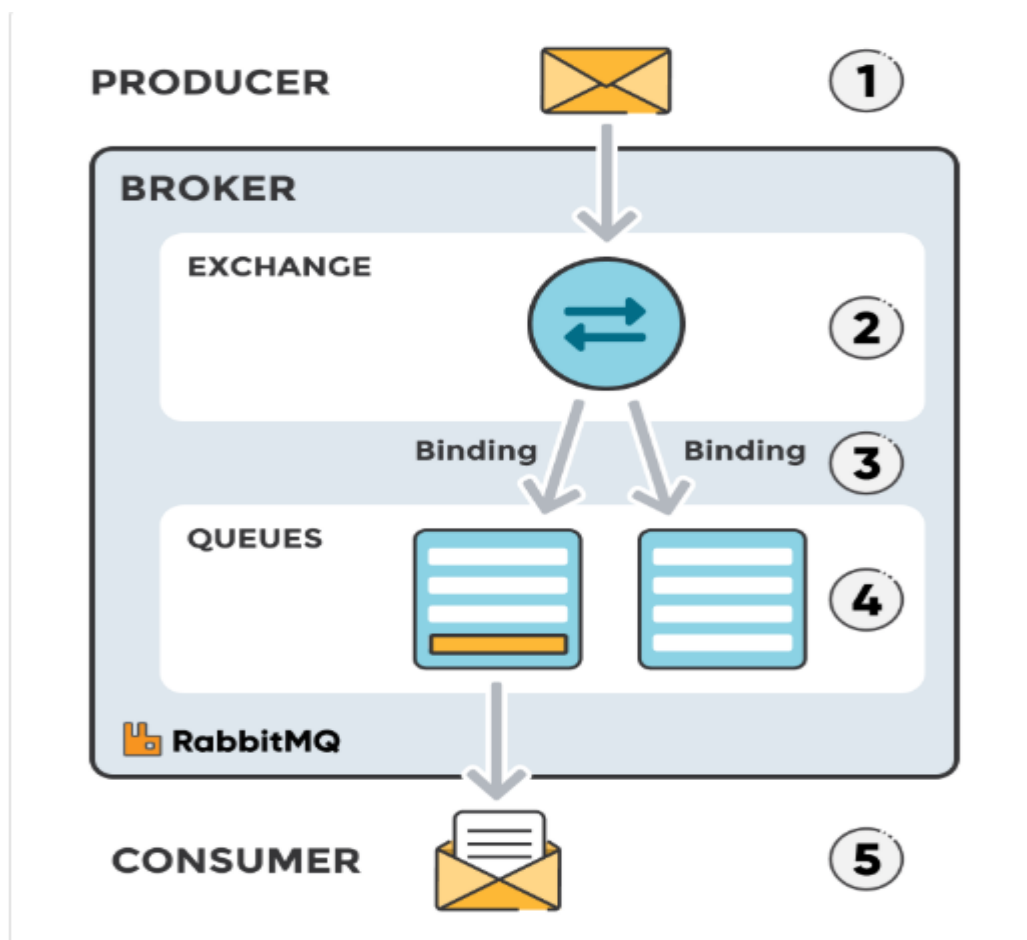
Комуникација путем RabbitMQ-а је **асинхрона**, што значи да сервер који генерише CRDT операцију не мора да чека да је остале серверске инстанце преузму и обраде. Операција се прослеђује систему за размену порука, након чега сервер може наставити са обрадом сопственог захтева, док остале инстанце операцију преузимају и обрађују независно. Истовремено, producer и consumer нису директно зависни један од другог, већ комуницирају посредством порука, чиме се смањује њихова међусобна зависност. На тај начин серверске инстанце не морају директно да комуницирају једна са другом нити да буду истовремено доступне како би размениле CRDT операције.

У RabbitMQ систему, поруке се не шаљу директно у queue, већ их producer најпре прослеђује **exchange** компоненти. Exchange је задужен за усмеравање порука ка једном или више queue-ова на основу свог типа и правила повезивања (bindings). Binding представља везу између exchange-а и queue-а. Након што порука буде прослеђена у queue, она остаје у њему све док је consumer не преузме и обради.

RabbitMQ подржава више типова exchange-а, међу којима су **Direct**, **Fanout**, **Topic** и **Headers**. Direct exchange усмерава поруку на основу тачног поклапања routing key-а и binding key-а, док Topic exchange омогућава усмеравање на основу шаблона routing key-а. Headers exchange користи вредности заглавља поруке, док Fanout exchange прослеђује поруку свим queue-овима који су повезани са њим.

У имплементираном систему користи се **Fanout exchange**, јер је потребно да свака CRDT операција буде доступна свим серверским инстанцама које учествују у раду над документом. Када једна инстанца генерише операцију, она је прослеђује exchange-у, који је затим дистрибуира свим повезаним queue-овима. Свака серверска инстанца има свој queue из којег преузима пристигле поруке и примењује добијене CRDT операције над локалном репликом документа.

Код Fanout exchange-а routing key нема утицај на избор queue-а, јер се порука без додатног филтрирања прослеђује свим повезаним queue-овима. Овакво понашање одговара потребама имплементираног система, јер све серверске реплике које учествују у раду над истим документом треба да добију исту CRDT операцију. На тај начин RabbitMQ обезбеђује асинхрону дистрибуцију операција, док CRDT механизам омогућава њихову доследну примену на различитим репликама. Пример размене порука путем RabbitMQ брокера приказан је на слици 5.1.



Слика 5.1. – Пример размене порука путем RabbitMQ брокера између producer-а и consumer-а.

5.3. WebSocket и SignalR комуникација

WebSocket представља комуникациони протокол који омогућава успостављање трајне, двосмерне везе између клијента и сервера преко једне TCP везе. За разлику од класичног HTTP модела, код којег клијент иницира захтев и чека одговор сервера, WebSocket омогућава да и клијент и сервер у било ком тренутку шаљу поруке другој страни. Након успостављања везе, комуникација се одвија без потребе за креирањем новог HTTP захтева за сваку поруку.

Успостављање WebSocket комуникације започиње HTTP захтевом којим клијент захтева прелазак са HTTP комуникације на WebSocket протокол. Уколико сервер прихвати захтев, веза се надограђује и успоставља се дуготрајна двосмерна комуникација. Након тога, подаци се могу размењивати у оба смера све док се веза не прекине.

Захваљујући оваквом приступу, обе стране могу да шаљу поруке једна другој у било ком тренутку, без потребе да свака размена података почиње новим захтевом, што значајно смањује кашњење и мрежно оптерећење у односу на понављано слање HTTP захтева. Ова особина чини WebSocket нарочито погодним за апликације које захтевају ажурирање у реалном времену, као што су системи за колаборативно уређивање докумената, где сервер

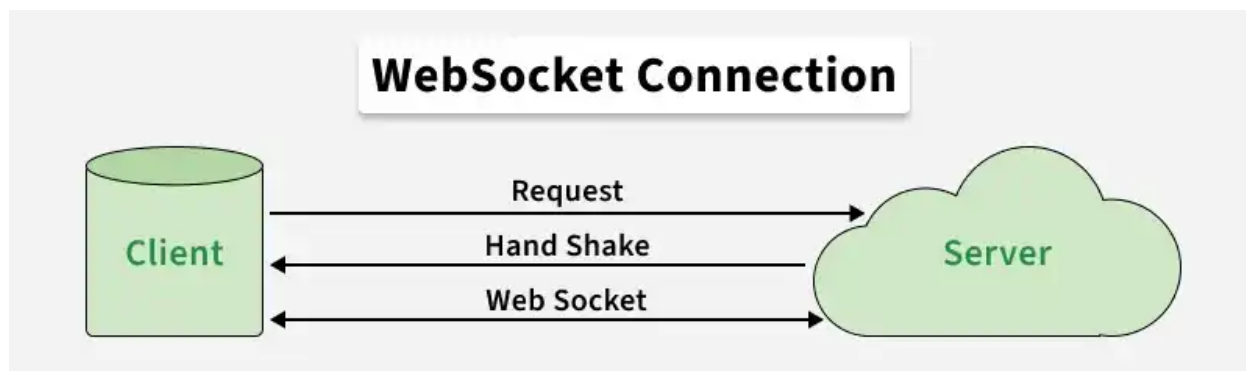
треба одмах да проследи измену свим повезаним клијентима чим до ње дође.

SignalR представља библиотеку у оквиру ASP.NET Core платформе која омогућава комуникацију у реалном времену између сервера и клијената, апстрахујући притом детаље транспортног протокола који се користи за пренос података. Иако SignalR у позадини по правилу користи WebSocket везу када год је она доступна, он аутоматски прелази на алтернативне механизме преноса, попут long polling, уколико WebSocket из неког разлога није подржан на страни клијента или мреже, чиме се обезбеђује да комуникација у реалном времену функционише.

Комуникација у SignalR-у заснива се на **Hub** компоненти. Hub представља посреднички слој између клијената и сервера преко којег се могу позивати серверске методе и слати поруке једном или већем броју повезаних клијената. Клијент успоставља везу са Hub-ом, након чега може слати податке серверу, док сервер може иницирати слање података клијенту.

У оквиру овог рада SignalR се користи за комуникацију између TinyMCE додатка и серверске апликације. Када корисник изврши измену текста, додаток формира одговарајућу CRDT операцију и прослеђује је серверу путем SignalR-а. Сервер затим обрађује примљену операцију и прослеђује је другим клијентима који уређују исти документ.

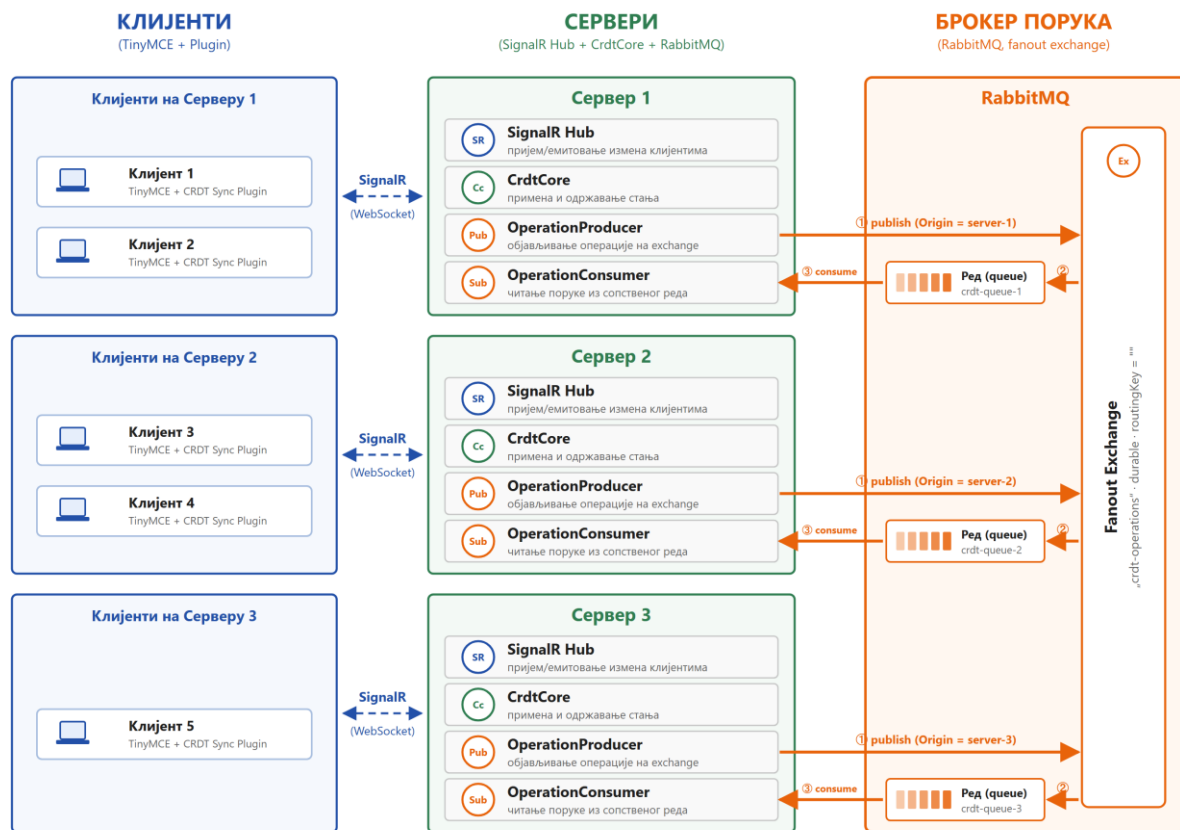
На овај начин SignalR омогућава да се измене једног корисника практично тренутно проследе осталим корисницима, што представља основу за реализацију колаборативног уређивања текста. Успостављање WebSocket везе између клијента и сервера приказано је на слици 5.2.



Слика 5.2. – Успостављање WebSocket везе између клијента и сервера.

5.4. Архитектура система

На слици 5.3. дат је приказ архитектуре система:



Слика 5.3. – Архитектура система.

Систем се састоји од клијентске стране, серверских инстанци и посредничког слоја за асинхрону комуникацију, реализованог коришћењем RabbitMQ-а. Сваки клијент представља инстанцу TinyMCE едитора проширену crdtsync додатком, који је задужен за превозње корисничких измена у операције над локалном CRDT репликом документа, као и за примену измена примљених са сервера. Клијенти се групишу према серверу на који су повезани, при чему поједини сервери могу опслуживати различит број клијената, што одражава ситуацију у којој оптерећење није равномерно распоређено међу серверима.

Комуникација између клијента и његовог сервера остварује се преко SignalR-а, при чему се, како је претходно описано, у позадини по правилу користи WebSocket веза. Ова веза је двосмерна – клијент серверу шаље измене настале локалним уређивањем документа, док сервер пристигле операције прослеђује осталим клијентима који су повезани на исти сервер.

Сваки сервер се, посматрано изнутра, састоји од три основне компоненте. **SignalR Hub** задужен је за пријем измена од повезаних клијената и њихово прослеђивање осталим клијентима на истом серверу. **CrdtCore** представља библиотеку класа која садржи имплементацију CRDT логике и управљање локалним стањем документа. **RabbitMQ комуникациони слој** задужен је за асинхронно прослеђивање локално насталих CRDT операција осталим серверским инстанцама. Сервер генерисану операцију шаље RabbitMQ

exchange-y, одакле се она прослеђује queue-овима повезаним са серверским репликама.

Оваква архитектура представља систем без лидера, у коме ниједан сервер нема посебан ауторитет над стањем документа нити улогу координатора над осталима, за разлику од класичних single-leader решења. Конвергенција свих реплика постиже се својствима саме CRDT структуре, док RabbitMQ обезбеђује асинхрону дистрибуцију операција између серверских инстанци. На овај начин комуникациони механизам и механизам за очување конвергенције имају јасно раздвојене улоге: RabbitMQ је задужен за пренос операција, док CRDT механизам одређује како се те операције примењују над локалним репликама.

5.5. Имплементација CRDT механизма

У наставку је детаљно приказана имплементација CRDT механизма кроз три компоненте система описане у претходном одељку. Прво је представљена CrdtCore библиотека, која садржи основну логику CRDT структуре података – представљање елемената текста, операције уметања и брисања, форматирање текста, као и механизам за постизање конвергенције независно од редоследа примене измена. Потом је описана CrdtServer апликација, која користи библиотеку CrdtCore, омогућава комуникацију са клијентима путем SignalR-а и користи RabbitMQ за дистрибуцију CRDT операција између серверских инстанци. На крају је приказана имплементација на страни клијента, односно начин на који TinyMCE додатак детектује измене настале уређивањем документа, преводи их у одговарајуће CRDT операције и размењује их са сервером.

5.5.1. CrdtCore библиотека

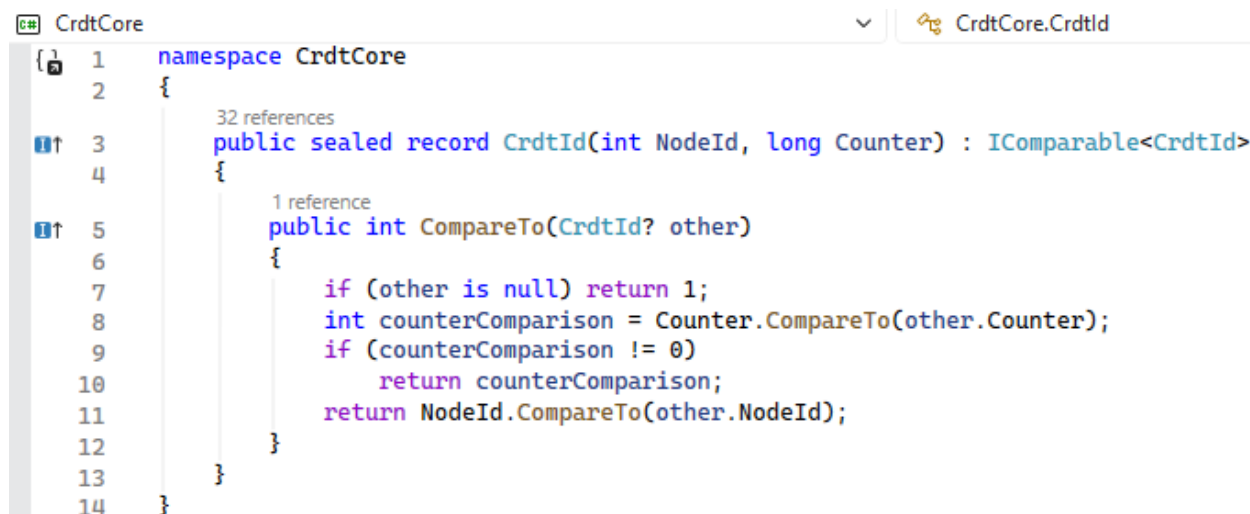
За потребе имплементације CRDT механизма развијена је посебна CrdtCore библиотека, која садржи основне структуре података и алгоритме неопходне за рад са CRDT документом. Издавањем CRDT логику у засебну библиотеку омогућено је њено одвајање од комуникационог слоја и корисничког интерфејса, чиме се исти механизам може користити у различитим деловима система без директне зависности од TinyMCE едитора или комуникационих технологија.

Библиотека представља језгро система за колаборативно уређивање текста. У њој су имплементирани механизми за представљање идентификатора и текстуалних елемената, управљање локалном репликом документа, уметање и брисање елемената, форматирање текста, као и обрада конкурентних измена. Комуникација са другим репликама није део саме CRDT структуре, већ се операције које библиотека производи могу проследити кроз комуникациони слој система. CrdtServer апликација користи ову библиотеку за креирање и управљање CRDT репликама докумената и обраду пристиглих операција.

Сваки елемент у CRDT документу идентификован је јединственим идентификатором који се састоји од два дела: NodeId и Counter. NodeId представља идентификатор чвора, односно клијента на којем је елемент креиран, док Counter представља локални бројач операција. Комбинација ова два параметра обезбеђује јединствену идентификацију елемената креираних на различитим чворовима.

CrdtId је имплементиран као рекорд (record) и реализује интерфејс IComparable<CrdtId>, чиме је омогућено дефинисање редоследа идентификатора. Приликом поређења најпре се пореде вредности локалних бројача (Counter). Уколико су вредности бројача различите, предност има идентификатор са мањом вредношћу бројача. У случају једнаких вредности бројача, као додатни критеријум користи се NodeId, чиме се обезбеђује детерминистички

редослед и у случају када су елементи креирани на различитим чворовима са истом вредношћу бројача. Приказ рекорда `CrdtId` дат је на слици 5.4.



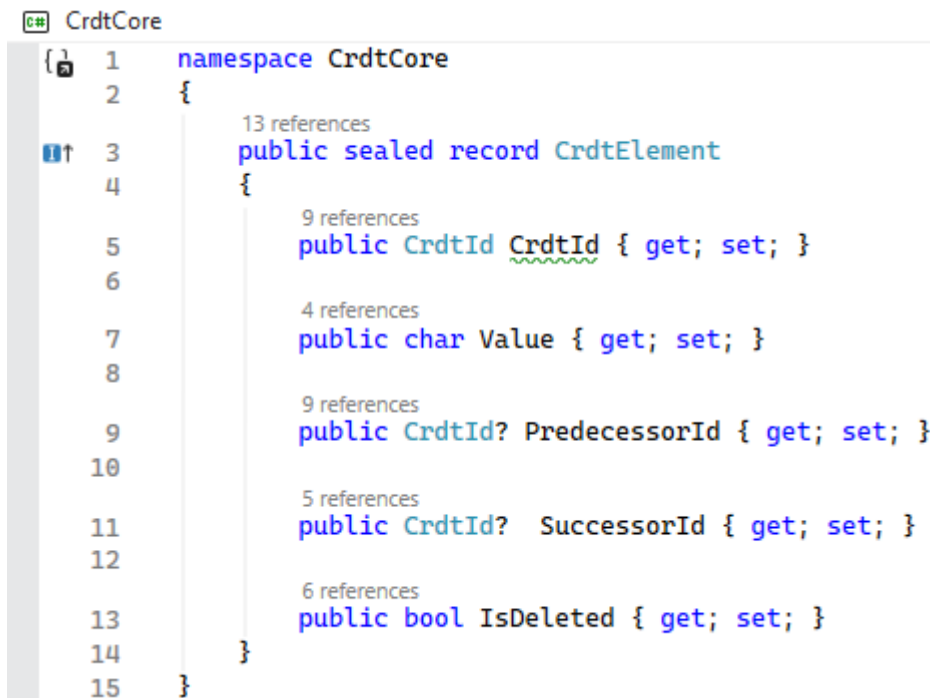
```
1 namespace CrdtCore
2 {
3     public sealed record CrdtId(int NodeId, long Counter) : IComparable<CrdtId>
4     {
5         public int CompareTo(CrdtId? other)
6         {
7             if (other is null) return 1;
8             int counterComparison = Counter.CompareTo(other.Counter);
9             if (counterComparison != 0)
10                 return counterComparison;
11             return NodeId.CompareTo(other.NodeId);
12         }
13     }
14 }
```

Слика 5.4. – Рекорд `CrdtId` који представља јединствени идентификатор сваког елемента у систему.

Појединачни елемент документа представљен је рекордом `CrdtElement`. Сваки елемент садржи јединствени идентификатор `CrdtId`, вредност коју представља, референцу на претходни и наредни елемент, као и ознаку која указује на то да ли је елемент логички обрисан.

Својство `CrdtId` одређује јединствени идентификатор елемента, док `Value` садржи конкретну вредност елемента, која је у имплементацији представљена једним знаком типа `char`. Својства `PredecessorId` и `SuccessorId` садрже идентификаторе претходног и наредног елемента у структури и омогућавају одређивање односа између елемената. Оба својства су nullable, јер први елемент може да нема претходника, док последњи елемент може да нема наредног елемента.

Својство `IsDeleted` користи се за означавање логички обрисаних елемената. Приликом брисања елемент се не уклања физички из структуре, већ се означава као обрисан. Рекорд `CrdtElement` приказан је на слици 5.5.



```

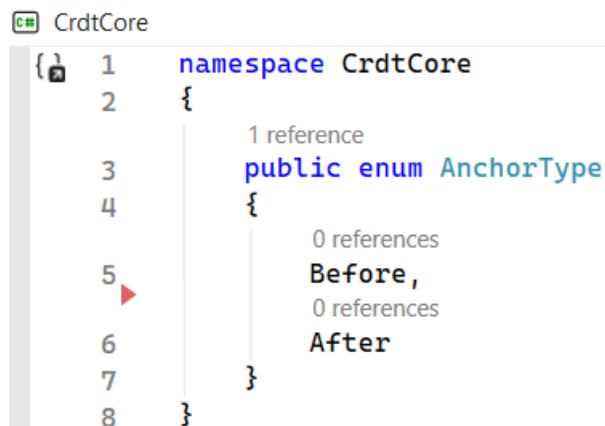
1 namespace CrdtCore
2 {
3     13 references
4     public sealed record CrdtElement
5     {
6         9 references
7         public CrdtId CrdtId { get; set; }
8
9         4 references
10        public char Value { get; set; }
11
12        9 references
13        public CrdtId? PredecessorId { get; set; }
14
15        5 references
16        public CrdtId? SuccessorId { get; set; }
17
18        6 references
19        public bool IsDeleted { get; set; }
20    }
21 }

```

Слика 5.5. – Репорд CrdtElement којим се представља сваки елемент у документу.

Поред операција над текстуалним елементима, у библиотеци су дефинисани и типови и механизми за представљање и примену форматирања текста. На слици 5.6. приказана је енумерација AnchorType, која дефинише две могуће вредности: Before, која означава позицију пре референтног елемента, и After, која означава позицију после референтног елемента. Репорд CrdtAnchor, приказан на слици 5.7, представља сидро (anchor) и садржи својства Id и Type. Својство Id представља идентификатор референтног елемента типа CrdtId, при чему може имати и вредност null, док својство Type одређује положај у односу на референтни елемент. Када је вредност својства Id једнака null, сидро означава почетак или крај документа, у зависности од вредности својства Type.

Ови типови омогућавају задавање граница опсега форматирања помоћу идентификатора елемената, уместо ослањања искључиво на нумеричке индексе у тексту. На тај начин границе форматирања могу се одредити у односу на тренутно стање CRDT структуре и након измена текста насталих током колаборативног уређивања.



```

1 namespace CrdtCore
2 {
3     1 reference
4     public enum AnchorType
5     {
6         0 references
7         Before,
8         0 references
9         After
10    }
11 }

```

Слика 5.6. – Енумерација AnchorType која дефинише тип позиције сидра у односу на референтни елемент.

```

namespace CrdtCore
{
    2 references
    public sealed record CrdtAnchor(CrdtId? Id, AnchorType Type);
}

```

Слика 5.7. – Рекорд CrdtAnchor који представља сидро за одређивање позиције у документу у односу на референтни елемент.

На слици 5.8. приказан је рекорд CrdtFormatting, који представља запис о форматирању одређеног опсега текста у CRDT документу. Својство FormattingId представља јединствени идентификатор записа о форматирању типа CrdtId, док својства Start и End представљају почетну и крајњу границу опсега форматирања и типа су CrdtAnchor. Својство Attributes представља речник парова назив–вредност којима се дефинишу атрибути форматирања. На овај начин се форматирање чува независно од самих текстуалних елемената и примењује на одређени опсег текста дефинисан помоћу сидара.

```

1  namespace CrdtCore
2  {
3      6 references
4      public sealed record CrdtFormatting
5      {
6          0 references
7          public CrdtId FormattingId { get; set; }
8
9          0 references
10         public CrdtAnchor Start { get; set; }
11
12         0 references
13         public CrdtAnchor End { get; set; }
14
15         0 references
16         public Dictionary<string, string> Attributes { get; set; }
17     }
18 }

```

Слика 5.8. – Рекорд CrdtFormatting који представља запис о форматирању одређеног опсега текста у CRDT документу.

Сам CRDT документ представљен је класом CrdtDocument, која садржи листу објеката типа CrdtElement, као и листу објеката типа CrdtFormatting који представљају записе о форматирању текста. Класа такође садржи привремене листе за елементе и операције брисања које чекају на обраду. Над документом су дефинисане основне операције за додавање и брисање елемената, додавање форматирања и примену операција насталих током рада у офлајн режиму. Класа CrdtDocument приказана је на слици 5.9.

```

1 namespace CrdtCore
2 {
3     20 references
4     public class CrdtDocument
5     {
6         17 references
7         public List<CrdtElement> Elements { get; set; } = [];
8
9         5 references
10        public List<CrdtFormatting> Formattings { get; set; } = [];
11
12        4 references
13        private readonly List<CrdtElement> _pendingElements = [];
14
15        4 references
16        private readonly List<CrdtId> _pendingDeletes = [];

```

Слика 5.9. – Класа CrdtDocument која представља CRDT документ и садржи текстуалне елементе, форматирања и листе операција које чекају на примену.

Операција брисања реализована је методом Delete, која на основу идентификатора CrdtId проналази елемент који треба обрисати. Уколико је елемент пронађен, његово својство IsDeleted поставља се на вредност true, чиме се елемент логички означава као обрисан, без његовог физичког уклањања из листе. Уколико елемент у тренутку примене операције није доступан, на пример услед мрежног кашњења, његов идентификатор се смешта у листу _pendingDeletes. На овај начин операција брисања може бити примењена накнадно, када одговарајући елемент постане доступан. Ово је значајно у колаборативном окружењу, јер операције могу стизати различитим редоследом услед мрежних кашњења и не морају бити примењене истим редоследом којим су настале. Метода Delete приказана је на слици 5.10.

```

111        8 references
112        public CrdtElement? Delete(CrdtId targetId)
113        {
114            var elementToDelete = Elements.FirstOrDefault(e => e.CrdtId == targetId);
115
116            if (elementToDelete == null)
117            {
118                _pendingDeletes.Add(targetId);
119                return null;
120            }
121
122            elementToDelete.IsDeleted = true;
123            return elementToDelete;
124        }

```

Слика 5.10. – Метода Delete којом се елементи логички означавају као обрисани.

Операција уметања реализована је методом Insert, која најпре покушава да уметне нови елемент на одговарајућу позицију позивом методе TryInsertElementInOrder. Уколико претходник новог елемента није присутан у документу, операција се не примењује одмах, већ се елемент смешта у листу _pendingElements и чека да одговарајући елемент постане доступан. Ово омогућава правилну обраду операција које због мрежног кашњења могу стићи различитим редоследом.

Метода TryInsertElementInOrder најпре проверава доступност претходника и одређује

почетну позицију за уметање. Уколико је дефинисан наредни елемент који је присутан у документу, његова позиција представља горњу границу претраге. Алгоритам затим пролази кроз елементе између претходника и наредног елемента и утврђује да ли припадају истој групи конкурентних уметања. За елементе који имају истог претходника, редослед се одређује поређењем њихових CrdtId вредности помоћу методе CompareTo, чиме се обезбеђује детерминистички редослед конкурентних уметања. Када се одреди одговарајућа позиција, нови елемент се уметне у листу Elements.

Након успешног уметања позива се метода ApplyPendingOperations, којом се проверавају операције које су претходно одложене због недоступних елемената и покушава њихова накнадна примена. Методе Insert и TryInsertElementInOrder приказане су на слици 5.11.

```

32 public CrdtElement Insert(CrdtElement crdtElement)
33 {
34     if (!TryInsertElementInOrder(crdtElement))
35     {
36         _pendingElements.Add(crdtElement);
37         return crdtElement;
38     }
39
40     ApplyPendingOperations();
41
42     return crdtElement;
43 }
44
45 private bool TryInsertElementInOrder(CrdtElement newElement)
46 {
47     if (newElement.PredecessorId != null && FindElementIndexById(newElement.PredecessorId) == -1)
48     {
49         return false;
50     }
51
52     var insertAfterIndex = FindElementIndexById(newElement.PredecessorId);
53     var candidateIndex = insertAfterIndex + 1;
54
55     var successorIndex = FindElementIndexById(newElement.SuccessorId);
56     var boundIndex = newElement.SuccessorId != null && successorIndex >= 0 ? successorIndex : Elements.Count;
57
58     var skippedIds = new HashSet<CrdtId?> { newElement.PredecessorId };
59
60     while (candidateIndex < boundIndex)
61     {
62         var candidate = Elements[candidateIndex];
63
64         bool partOfConflicts = skippedIds.Contains(candidate.PredecessorId);
65
66         if (!partOfConflicts)
67             break;
68
69         if (candidate.PredecessorId == newElement.PredecessorId
70             && candidate.CrdtId.CompareTo(newElement.CrdtId) <= 0) // exit when new element has priority
71             break;
72
73         skippedIds.Add(candidate.CrdtId);
74         candidateIndex++;
75     }
76
77     Elements.Insert(candidateIndex, newElement);
78
79     return true;
80 }

```

Слика 5.11. – Методе Insert и TryInsertElementInOrder којима се елемент уметне у структуру.

На слици 5.12. приказане су методе ApplyPendingOperations и ApplyFormatting. Метода ApplyPendingOperations користи се за накнадну примену операција које претходно нису могле бити извршене због недоступности одговарајућих елемената. У оквиру петље, метода покушава да поново примени све операције уметања и брисања које се налазе у листама _pendingElements и _pendingDeletes. Успешно примењене операције уклањају се

из одговарајућих листа, а поступак се понавља све док се у једном пролазу примењује бар једна одложена операција. На овај начин могу се обрадити и операције које постану извршиве тек након примене претходно одложених операција. Метода `ApplyFormatting` примљени објекат типа `CrdtFormatting` додаје у листу `Formattings`, чиме се форматирање чува као део стања CRDT документа и може се применити на одговарајући опсег текста.

```

83  ✓      1 reference
      private void ApplyPendingOperations()
84      {
85          bool anyChanged;
86  ✓      do
87      {
88          anyChanged = false;
89
90  ✓      for (int i = _pendingElements.Count - 1; i >= 0; i--)
91      {
92          var pendingElement = _pendingElements[i];
93  ✓      if (TryInsertElementInOrder(pendingElement))
94      {
95          _pendingElements.RemoveAt(i);
96          anyChanged = true;
97      }
98      }
99
100  ✓      for (int i = _pendingDeletes.Count - 1; i >= 0; i--)
101      {
102  ✓      if (TryDeleteElement(_pendingDeletes[i]))
103      {
104          _pendingDeletes.RemoveAt(i);
105          anyChanged = true;
106      }
107      }
108      } while (anyChanged);
109  }
      3 references
110  ✓  public CrdtFormatting ApplyFormatting(CrdtFormatting formatting)
111  {
112      Formattings.Add(formatting);
113      return formatting;
114  }

```

Слика 5.12. – Методе `ApplyPendingOperations` и `ApplyFormatting` за накнадну примену одложених операција и додавање форматирања у CRDT документ.

За потребе тестирања `CrdtCore` библиотеке креиран је посебан тест пројекат `CrdtCore.Tests`, који садржи укупно седам тестова. Тестовима се проверавају основне операције уметања и брисања, као и понашање система у различитим ситуацијама. Посебно су значајни тестови који проверавају обраду конкурентних измена и конвергенцију реплика. Ови тестови симулирају ситуације у којима различите реплике примају исте операције различитим редоследом. Након обраде свих операција, упоређује се коначно стање документа како би се проверило да ли реплике достижу исто стање независно од редоследа пристиглих измена.

Најзначајнији тест у оквиру тест пројекта је `Insert_DifferentInsertionOrdersFromMultipleNodes_ConvergeToSameState`. Њиме се

симулира рад три реплике документа које примају исте конкурентне операције уметања, али различитим редоследом. Све операције се односе на елементе који имају исти претходни елемент, али потичу са различитих чворова.

Након примене операција на све три реплике, њихова коначна стања се међусобно упоређују. Тест проверава да ли све реплике достижу исто стање независно од редоследа којим су операције примењене. Успешно извршавање овог теста потврђује једно од кључних својстава имплементираног CRDT механизма – конвергенцију реплика при конкурентним изменама. Тест је приказан на слици 5.13.

```
[Fact]
0 references
public void Insert_DifferentInsertionOrdersFromMultipleNodes_ConvergeToSameState()
{
    //Arrange
    var idA = new CrdtId(1, 0);
    var idB = new CrdtId(1, 1);
    var idC = new CrdtId(2, 2);
    var idD = new CrdtId(3, 3);

    var firstDoc = new CrdtDocument();
    firstDoc.Insert(Element(idA, 'A'));

    var secondDoc = new CrdtDocument();
    secondDoc.Insert(Element(idA, 'A'));

    var thirdDoc = new CrdtDocument();
    thirdDoc.Insert(Element(idA, 'A'));

    //Act
    firstDoc.Insert(Element(idB, 'B', idA));
    firstDoc.Insert(Element(idC, 'C', idA));
    firstDoc.Insert(Element(idD, 'D', idA));

    secondDoc.Insert(Element(idC, 'C', idA));
    secondDoc.Insert(Element(idB, 'B', idA));
    secondDoc.Insert(Element(idD, 'D', idA));

    thirdDoc.Insert(Element(idD, 'D', idA));
    thirdDoc.Insert(Element(idC, 'C', idA));
    thirdDoc.Insert(Element(idB, 'B', idA));

    //Assert
    Assert.Equal(firstDoc.GetText(), secondDoc.GetText());
    Assert.Equal(firstDoc.GetText(), thirdDoc.GetText());
    Assert.Equal(secondDoc.GetText(), thirdDoc.GetText());
}
```

Слика 5.13. – Тестирање конвергенције три реплике документа које, након примене конкурентних операција различитим редоследом, достижу идентично коначно стање.

Такође је значајан тест `Insert_ConcurrentInsertsAtSamePosition_ConvergeRegardlessOfOrder`, који проверава понашање система када два различита чвора истовремено умећу елементе на исту позицију у документу. У тесту оба нова елемента имају исти `PredecessorId`, што значи да се умећу иза истог елемента.

Операције се на две реплике примењују различитим редоследом: на једној реплици прво се умеће елемент В, а затим С, док се на другој прво умеће С, а затим В. На крају се упоређује текст обе реплике. Успешно извршавање теста потврђује да алгоритам обезбеђује исто коначно стање независно од редоследа пристизања конкурентних

операција. Тест је приказан на слици 5.14.

```
[Fact]
0 references
public void Insert_ConcurrentInsertsAtSamePosition_ConvergeRegardlessOfOrder()
{
    //Arrange
    var idA = new CrdtId(1, 0);
    var idB = new CrdtId(1, 1);
    var idC = new CrdtId(2, 2);

    var docBFirst = new CrdtDocument();
    var docCFirst = new CrdtDocument();

    //Act
    docBFirst.Insert(Element(idA, 'A'));
    docBFirst.Insert(Element(idB, 'B', idA));
    docBFirst.Insert(Element(idC, 'C', idA));

    docCFirst.Insert(Element(idA, 'A'));
    docCFirst.Insert(Element(idC, 'C', idA));
    docCFirst.Insert(Element(idB, 'B', idA));

    //Assert
    Assert.Equal(docBFirst.GetText(), docCFirst.GetText());
}
```

Слика 5.14. – Тестирање конкурентних операција уметања на исту позицију, при чему обе реплике достижу исто коначно стање независно од редоследа примене операција.

5.5.2. CrdtServer

CrdtServer представља веб сервер који користи функционалности CrdtCore библиотеке за рад са CRDT документима. У оквиру сервера интегрисане су потребне библиотеке за реализацију комуникације, укључујући SignalR и библиотеке неопходне за комуникацију са RabbitMQ системом за размену порука.

Свака серверска инстанца поседује сопствену локалну копију CRDT документа, која се током рада учитава и чува у оквиру саме инстанце. На овај начин серверске инстанце не деле исту инстанцу документа, већ свака од њих самостално одржава своју реплику. За трајно чување локалне копије документа користе се JSON датотеке, док се синхронизација између различитих серверских инстанци реализује разменом CRDT операција путем RabbitMQ система.

Када сервер прими операцију од клијента, операција се најпре примењује на локалну реплику документа, а затим се прослеђује путем RabbitMQ-а како би је остале серверске инстанце могле примити и применити на својим репликама. На овај начин се омогућава дистрибуција операција између више серверских инстанци без директне комуникације између њих.

Комуникација са клијентима реализована је помоћу SignalR-а, који омогућава двосмерну комуникацију у реалном времену. Клијенти преко SignalR везе шаљу CRDT операције серверу и примају операције које су настале на другим клијентима. RabbitMQ се користи као посредник за размену операција између серверских инстанци, чиме се комуникација са клијентима и комуникација између сервера одвајају у засебне слојеве.

Класа `CrdtDocumentStore` задужена је за чување и учитавање локалних копија CRDT докумената. За сваки документ користи се JSON датотека у директоријуму који се одређује на основу идентификатора серверске инстанце, чиме се омогућава да свака инстанца има сопствену копију документа. Метода `Save` серијализује тренутно стање документа и уписује га у одговарајућу JSON датотеку. Класа `CrdtDocumentStore` приказана је на слици 5.15.

```

7      public class CrdtDocumentStore
8      {
9          2 references
10         private readonly ConcurrentDictionary<string, CrdtDocument> _documents = new();
11         4 references
12         private readonly string _dataDirectory;
13
14         0 references
15         public CrdtDocumentStore(IConfiguration configuration)
16         {
17             var instanceId = configuration["RabbitMq:InstanceId"] ?? "default";
18             _dataDirectory = $"Data-{instanceId}";
19         }
20
21         5 references
22         public async Task Save(string docId, CrdtDocument document)
23         {
24             Directory.CreateDirectory(_dataDirectory);
25
26             var path = Path.Combine(_dataDirectory, $"{docId}.json");
27
28             string json;
29             lock (document)
30             {
31                 json = JsonSerializer.Serialize(document);
32             }
33
34             await File.WriteAllTextAsync(path, json);
35         }
36     }

```

Слика 5.15. – Класа `CrdtDocumentStore` и метода `Save` за чување локалне копије CRDT документа у JSON датотеци.

На слици 5.16. приказана је класа `RabbitMqConnection`, која је задужена за успостављање и управљање везом са RabbitMQ сервером. Приликом иницијализације, адреса и порт сервера учитавају се из конфигурације, након чега се помоћу `ConnectionFactory` успоставља веза. Класа имплементира `IDisposable` како би се омогућило ослобађање ресурса након завршетка рада.

```

5 public class RabbitMqConnection : IRabbitMqConnection, IDisposable
6 {
7     3 references
8     private IConnection? _connection { get; set; }
9
10    3 references
11    public IConnection Connection => _connection!;
12
13    3 references
14    public IConfiguration _configuration;
15
16    0 references
17    public RabbitMqConnection(IConfiguration configuration)
18    {
19        _configuration = configuration;
20        _connection = InitializeConnection().GetAwaiter().GetResult();
21    }
22
23    1 reference
24    private async Task<IConnection> InitializeConnection()
25    {
26        var factory = new ConnectionFactory
27        {
28            HostName = _configuration["RabbitMq:HostName"] ?? "localhost",
29            Port = int.TryParse(_configuration["RabbitMq:Port"], out var port) ? port : 5672
30        };
31
32        return await factory.CreateConnectionAsync();
33    }
34
35    0 references
36    public void Dispose()
37    {
38        _connection!.Dispose();
39    }
40 }

```

Слика 5.16. – Класа RabbitMqConnection за успостављање и управљање везом са RabbitMQ сервером.

На слици 5.17. приказана је класа OperationProducer, која је задужена за слање CRDT операција путем RabbitMQ-а. Метода SendOperation успоставља канал, креира трајни fanout exchange и операцију смешта у поруку која садржи тип операције, идентификатор документа, идентификатор серверске инстанце и податке операције. Порука се затим серијализује у JSON формат и прослеђује на exchange, одакле је преузимају остале серверске инстанце.

```

1 reference
8 public class OperationProducer(IRabbitMqConnection connection, IConfiguration configuration) : IOperationProducer
9 {
10    7 references
11    public async Task SendOperation<T>(string type, string docId, T data)
12    {
13        using var channel = await connection.Connection.CreateChannelAsync();
14        await channel.ExchangeDeclareAsync("crdt-operations", ExchangeType.Fanout, durable: true);
15
16        var envelope = new { Type = type, DocId = docId, Origin = configuration["RabbitMq:InstanceId"], Data = data };
17
18        var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(envelope));
19
20        await channel.BasicPublishAsync(exchange: "crdt-operations", routingKey: "", body: body);
21    }
22 }
23 }

```

Слика 5.17. – Класа OperationProducer за слање CRDT операција путем RabbitMQ-а.

На слици 5.18. приказан је део класе `OperationConsumer`, која је имплементирана као `BackgroundService` и задужена за пријем CRDT операција путем RabbitMQ-а. Приликом покретања креира се канал и конфигурише `fanout exchange crdt-operations`. За сваку серверску инстанцу креира се посебан `queue`, чији назив садржи идентификатор инстанце, а затим се `queue` повезује са одговарајућим `exchange`-ом. На тај начин свака серверска инстанца прима сопствену копију порука послатих на `exchange`. За обраду пристиглих порука користи се `AsyncEventingBasicConsumer`, док се пријем порука покреће без аутоматске потврде њиховог пријема (`autoAck: false`).

```

1 reference
10 public class OperationConsumer(IRabbitMqConnection connection,
11     IConfiguration configuration,
12     CrdtDocumentStore store,
13     IHubContext<CrdtHub> hubContext) : BackgroundService
14 {
15
16     2 references
17     private readonly string _instanceId = configuration["RabbitMq:InstanceId"]!;
18
19     0 references
20     protected override async Task ExecuteAsync(CancellationToken stoppingToken)
21     {
22         var channel = await connection.Connection.CreateChannelAsync(cancellationToken: stoppingToken);
23         await channel.ExchangeDeclareAsync("crdt-operations", ExchangeType.Fanout, durable: true);
24         var queue = $"crdt-queue-{_instanceId}";
25
26         await channel.QueueDeclareAsync(queue: queue,
27             durable: true, exclusive: false,
28             autoDelete: false,
29             cancellationToken: stoppingToken);
30         await channel.QueueBindAsync(queue,
31             exchange: "crdt-operations",
32             routingKey: "",
33             cancellationToken: stoppingToken);
34
35         var consumer = new AsyncEventingBasicConsumer(channel);
36         consumer.ReceivedAsync += async (_, ea) => await HandleMessageAsync(channel, ea);
37
38         await channel.BasicConsumeAsync(queue, autoAck: false, consumer, cancellationToken: stoppingToken);
39         await Task.Delay(Timeout.Infinite, stoppingToken);
40     }
41

```

Слика 5.18. – Део класе `OperationConsumer` за конфигурисање RabbitMQ consumer-а и пријем CRDT операција.

На слици 5.19. приказане су методе `HandleMessageAsync` и `ApplyAsync`, које обрађују CRDT операције примљене путем RabbitMQ-а. Метода `HandleMessageAsync` десеријализује примљену поруку у објекат `OperationEnvelope` и проверава да ли је операција настала на другој серверској инстанци. Уколико јесте, прослеђује је методи `ApplyAsync`, након чега се порука потврђује позивом `BasicAckAsync`.

Метода `ApplyAsync` учитава локалну копију документа и, у зависности од типа операције, примењује операцију уметања, брисања или форматирања. Након измене документа, ново стање се прослеђује клијентима који су повезани на одговарајућу SignalR групу, а локална копија документа се затим чува у JSON датотеци.

```

42     private async Task HandleMessageAsync(IChannel channel, BasicDeliverEventArgs ea)
43     {
44         var envelope = JsonSerializer.Deserialize<OperationEnvelope>(ea.Body.Span, JsonSerializerOptions);
45
46         if(envelope != null && envelope.Origin != _instanceId)
47         {
48             await ApplyAsync(envelope);
49         }
50
51         await channel.BasicAckAsync(ea.DeliveryTag, multiple: false);
52     }
53
54     1 reference
55     private async Task ApplyAsync(OperationEnvelope envelope)
56     {
57         var document = await store.GetOrCreate(envelope.DocId);
58
59         switch (envelope.Type)
60         {
61             case "Insert":
62                 var element = envelope.Data.Deserialize<CrdtCore.CrdtElement>(JsonSerializerOptions);
63                 if (element != null)
64                 {
65                     document.Insert(element);
66                     await hubContext.Clients.Group(envelope.DocId).SendAsync("ElementsChanged", document.Elements);
67                 }
68                 break;
69             case "Delete":
70                 var id = envelope.Data.Deserialize<CrdtCore.CrdtId>(JsonSerializerOptions);
71                 if (id != null)
72                 {
73                     document.Delete(id);
74                     await hubContext.Clients.Group(envelope.DocId).SendAsync("ElementsChanged", document.Elements);
75                 }
76                 break;
77             case "Formatting":
78                 var formatting = envelope.Data.Deserialize<CrdtCore.CrdtFormatting>(JsonSerializerOptions);
79                 if (formatting != null)
80                 {
81                     document.ApplyFormatting(formatting);
82                     await hubContext.Clients.Group(envelope.DocId).SendAsync("FormattingsChanged", document.Formattings);
83                 }
84                 break;
85             }
86
87         await store.Save(envelope.DocId, document);
88     }
89

```

Слика 5.19. – Методе `HandleMessageAsync` и `ApplyAsync` за обраду пристиглих CRDT операција и ажурирање локалне реплике документа.

На слици 5.20. приказане су методе `JoinDocument`, `Insert`, `Delete` и `ApplyFormatting` класе `CrdtHub`, које реализују комуникацију између клијента и сервера путем `SignalR`-а. Метода `JoinDocument` додаје клијента у групу која одговара идентификатору документа, учитава локалну копију документа и клијенту прослеђује тренутне елементе и форматирања. Методе `Insert`, `Delete` и `ApplyFormatting` преузимају одговарајуће CRDT операције, примењују их над локалном копијом документа и чувају ажурирано стање документа.

Након примене операције, ново стање документа се путем `SignalR`-а прослеђује осталим клијентима у одговарајућој групи, при чему се клијенту који је послао операцију не шаље поново. Истовремено се операција прослеђује `OperationProducer` компоненти, која је шаље путем `RabbitMQ`-а како би је остале серверске инстанце могле применити на својим локалним репликама. На тај начин `CrdtHub` повезује комуникацију са клијентима путем `SignalR`-а са дистрибуцијом CRDT операција између серверских инстанци.

```

2 references
6 public class CrdtHub(CrdtDocumentStore store, IOperationProducer producer) : Hub
7 {
    3 references
8     private static readonly JsonSerializerOptions OfflineOperationJsonOptions = new()
9     {
10         PropertyNameCaseInsensitive = true,
11     };
12
13     0 references
14     public async Task JoinDocument(string docId)
15     {
16         await Groups.AddToGroupAsync(Context.ConnectionId, docId);
17         var document = await store.GetOrCreate(docId);
18         await Clients.Caller.SendAsync("ElementsChanged", document.Elements);
19         await Clients.Caller.SendAsync("FormattingsChanged", document.Formattings);
20     }
21
22     0 references
23     public async Task Insert(CrdtCore.CrdtElement crdtElement, string docId)
24     {
25         var document = await store.GetOrCreate(docId);
26         document.Insert(crdtElement);
27         await store.Save(docId, document);
28         await Clients.GroupExcept(docId, Context.ConnectionId).SendAsync("ElementsChanged", document.Elements);
29         await producer.SendOperation("Insert", docId, crdtElement);
30     }
31
32     0 references
33     public async Task Delete(CrdtCore.CrdtId crdtId, string docId)
34     {
35         var document = await store.GetOrCreate(docId);
36         document.Delete(crdtId);
37         await store.Save(docId, document);
38         await Clients.GroupExcept(docId, Context.ConnectionId).SendAsync("ElementsChanged", document.Elements);
39         await producer.SendOperation("Delete", docId, crdtId);
40     }
41
42     0 references
43     public async Task ApplyFormatting(CrdtCore.CrdtFormatting formatting, string docId)
44     {
45         var document = await store.GetOrCreate(docId);
46         document.ApplyFormatting(formatting);
47         await store.Save(docId, document);
48         await Clients.GroupExcept(docId, Context.ConnectionId).SendAsync("FormattingsChanged", document.Formattings);
49         await producer.SendOperation("Formatting", docId, formatting);
50     }
51
52     0 references
53     public async Task ApplyOfflineOperations(CrdtCore.CrdtOfflineOperation[] operations, string docId)
54     {
55         var document = await store.GetOrCreate(docId);
56         document.ApplyOfflineOperations(operations);
57         await store.Save(docId, document);
58         await Clients.GroupExcept(docId, Context.ConnectionId).SendAsync("ElementsChanged", document.Elements);
59         await producer.SendOperation("ApplyOfflineOperations", docId, operations);
60     }
61 }

```

Слика 5.20. – Методе JoinDocument, Insert, Delete и ApplyFormatting класе CrdtHub за комуникацију са клијентима и обраду CRDT операција путем SignalR-a.

На слици 5.21. приказана је метода ApplyOfflineOperations, која омогућава обраду CRDT операција насталих током рада клијента у офлајн режиму. Метода најпре учитава локалну копију документа, а затим за сваку пристиглу операцију на основу њеног типа десеријализује одговарајуће податке и примењује операцију над документом. Подржане су операције уметања, брисања и форматирања, при чему се свака успешно обрађена операција прослеђује OperationProducer компоненти како би била дистрибуирана осталим серверским инстанцама путем RabbitMQ-a.

Након обраде свих операција, ажурирано стање документа се чува, а ново стање елемената и форматирања прослеђује се свим клијентима који су повезани на одговарајућу SignalR групу. На овај начин се операције настале током офлајн рада накнадно примењују на серверу и укључују у даљу синхронизацију документа.

```

59 public async Task ApplyOfflineOperations(List<CrdtCore.OfflineOperations> operations, string docId)
60 {
61     var document = await store.GetOrCreate(docId);
62
63     foreach (var operation in operations)
64     {
65         switch (operation.Type)
66         {
67             case "Insert":
68                 var insertElement = JsonSerializer.Deserialize<CrdtCore.CrdtElement>(operation.Data.GetRawText(), OfflineOperationJsonOptions);
69                 if (insertElement != null)
70                 {
71                     document.Insert(insertElement);
72                     await producer.SendOperation("Insert", docId, insertElement);
73                 }
74                 break;
75             case "Delete":
76                 var deleteId = JsonSerializer.Deserialize<CrdtCore.CrdtId>(operation.Data.GetRawText(), OfflineOperationJsonOptions);
77                 if (deleteId != null)
78                 {
79                     document.Delete(deleteId);
80                     await producer.SendOperation("Delete", docId, deleteId);
81                 }
82                 break;
83             case "Formatting":
84                 var formatting = JsonSerializer.Deserialize<CrdtCore.CrdtFormatting>(operation.Data.GetRawText(), OfflineOperationJsonOptions);
85                 if (formatting != null)
86                 {
87                     document.ApplyFormatting(formatting);
88                     await producer.SendOperation("Formatting", docId, formatting);
89                 }
90                 break;
91         }
92     }
93
94     await store.Save(docId, document);
95
96     await Clients.Group(docId).SendAsync("ElementsChanged", document.Elements);
97     await Clients.Group(docId).SendAsync("FormattingsChanged", document.Formattings);
98 }
99 }

```

Слика 5.21. – Метода `ApplyOfflineOperations` за обраду операција насталих током рада у офлајн режиму и њихово прослеђивање осталим серверским инстанцама путем RabbitMQ-а.

5.5.3. Клијентска страна и TinyMCE додатак

TinyMCE омогућава проширивање функционалности едитора путем додатака (plugin-ова). Допатак се региструје коришћењем PluginManager API-ја, који представља улазну тачку за повезивање додатка са TinyMCE едитором. На овај начин додатак може приступити API-ју едитора и реаговати на догађаје који настају током уређивања документа. За препознавање прилагођеног додатка неопходно је да постоји JavaScript датотека која садржи његову главну улазну тачку и регистрацију путем PluginManager.add() методе. Приликом регистрације додатку се додељује јединствени идентификатор, док се сама логика додатка може организовати у више датотека.

Након креирања и регистрације додатка, потребно га је укључити у конкретну инстанцу TinyMCE едитора. Ово се врши приликом иницијализације едитора коришћењем функције `tinymce.init()`. У оквиру конфигурације, опција `external_plugins` користи се за задавање путање до JavaScript датотеке која садржи имплементацију додатка, док се у оквиру опције `plugins` наводи његов идентификатор. Поред тога, приликом иницијализације потребно је проследити и конфигурационе параметре које додатак очекује. У конкретној имплементацији то су `crdtsync_hub_url`, који представља адресу SignalR Hub-a, и `crdtsync_doc_id`, који одређује идентификатор документа са којим је клијент повезан. На тај начин додатак добија све неопходне параметре за успостављање комуникације са сервером и синхронизацију одговарајућег документа. Приказ иницијализације едитора и конфигурације додатка дат је на слици 5.22.

```

<body>
  <textarea id="editor"></textarea>
  <script>
    const serverPort =
      new URLSearchParams(location.search).get("port") || "7160";

    tinymce.init({
      selector: "#editor",
      height: 500,
      menubar: false,
      branding: false,
      promotion: false,
      external_plugins: {
        crdtsync: "/crdtsync.js",
      },
      plugins: "crdtsync",
      content_style:
        "body { white-space: pre-wrap; } .crdt-tab { display: inline-block; width: 2em; }",
      crdtsync_hub_url: `https://localhost:${serverPort}/editorHub`,
      crdtsync_doc_id: "demo",
    });
  </script>
</body>

```

Слика 5.22. – Приказ иницијализације TinyMCE едитора са crdtsync додатком и конфигурацијом URL адресе за повезивање са CRDT сервером и идентификатора документа.

На слици 5.23. приказано је успостављање SignalR везе између TinyMCE додатка и CrdtServer апликације. Веза се креира помоћу HubConnectionBuilder, при чему је омогућено аутоматско поновно повезивање у случају прекида комуникације. Приликом прекида везе кориснику се приказује обавештење да је могућ наставак рада у офлајн режиму.

Након поновног успостављања везе, клијент се поново придружује одговарајућем документу позивом методе JoinDocument, након чега се позива flushOfflineQueue како би се серверу проследили операције настале током прекида везе. На овај начин се омогућава наставак рада без губитка локално насталих измена.

```

editor.on("init", () => {
  connection = new signalR.HubConnectionBuilder()
    .withUrl(hubUrl)
    .withAutomaticReconnect({
      nextRetryDelayInMilliseconds: (retryContext) =>
        Math.min(30000, (retryContext.previousRetryCount + 1) * 2000),
    })
    .build();

  connection.onreconnecting((error) => {
    editor.notificationManager.open({
      text: "The server is currently unavailable. You can continue editing in offline mode.",
      type: "warning",
      timeout: 0,
    });
  });

  connection.onreconnected(async () => {
    editor.notificationManager.open({
      text: "Connection to the server has been restored.",
      type: "success",
      timeout: 3000,
    });

    await connection.invoke("JoinDocument", docId);
    await flushOfflineQueue();
  });

  connection
    .start()
    .then(() => connection.invoke("JoinDocument", docId))
    .catch((err) => console.error("[crdtsync] error during connection", err));
}

```

Слика 5.23. – Успостављање SignalR везе са сервером и обрада прекида и поновног успостављања везе.

На слици 5.24. приказана је обрада измена насталих у TinyMCE едитору. Приликом сваке измене најпре се добија тренутни текст документа и пореди са претходно сачуваним стањем помоћу функције `diffText`. На основу резултата поређења одређују се позиција измене, обрисани и уметнути текст. Уколико је дошло до брисања, одговарајући елементи се логички означавају као обрисани и генеришу се Delete операције.

За сваки нови карактер креира се `CrdtElement` са јединственим `CrdtId` идентификатором, који се формира на основу идентификатора клијента (`myNodeId`) и локалног бројача (`myCounter`). Поред тога, одређују се `predecessorId` и `successorId`, који дефинишу позицију новог елемента у CRDT структури. Нови елемент се додаје у локалну реплику, а одговарајућа Insert операција се прослеђује функцији `sendOrQueue` ради слања серверу или смештања у офлајн ред уколико веза није доступна.

```

editor.on("input", () => {
  if (isApplyingRemoteChange) {
    return;
  }

  const newText = editor
    .getContent({ format: "text" })
    .replace(/\n+/g, (run) => "\n".repeat(run.length / 2));
  const oldText = renderText();

  const { start, deleted, inserted } = diffText(oldText, newText);

  for (let i = 0; i < deleted.length; i++) {
    const el = findVisibleElementAt(localElements, start);
    if (!el) continue;
    el.isDeleted = true;
    sendOrQueue({
      type: "Delete",
      data: el.crdtId,
    });
  }

  for (let i = 0; i < inserted.length; i++) {
    const predecessorId = findPredecessorId(start + i);
    const successorElement = findVisibleElementAt(localElements, start + i);
    const successorId = successorElement ? successorElement.crdtId : null;
    const newElement = {
      crdtId: { nodeId: myNodeId, counter: myCounter++ },
      value: inserted[i],
      predecessorId,
      successorId,
      isDeleted: false,
    };

    const insertAt = predecessorId
      ? localElements.findIndex((e) => idsEqual(e.crdtId, predecessorId)) + 1
      : 0;
    localElements.splice(insertAt, 0, newElement);

    if (!firstNewElement) firstNewElement = newElement;
    lastNewElement = newElement;

    sendOrQueue({
      type: "Insert",
      data: newElement,
    });
  }
}

```

Слика 5.24. – Обрада измена текста у TinyMCE едитору и генерисање CRDT операција уметања и брисања.

На слици 5.25. приказана је обрада команди за форматирање текста у TinyMCE едитору. На основу тренутне селекције одређују се почетна и крајња позиција текста, које се затим помоћу функције `getAnchors` преводе у `CrdtAnchor` објекте. На овај начин се опсег форматирања одређује у односу на идентификаторе CRDT елемената, а не само на основу њихових тренутних позиција у тексту.

Након одређивања опсега, функција `getFormattingAttribute` идентификује тип форматирања и његову вредност, након чега се креира објекат `CrdtFormatting` са јединственим идентификатором, почетним и крајњим сидром и атрибутима форматирања. Креирано форматирање се додаје у локалну реплику и прослеђује функцији `sendOrQueue`, која га шаље серверу или смешта у ред за каснију синхронизацију уколико веза није доступна. У случају да није изабран опсег текста, атрибути се привремено чувају у `pendingAttributes` и примењују на наредно уметнути текст.

```
editor.on("ExecCommand", (e) => {
  const indexes = getSelectionIndexes();

  const startElement = findVisibleElementAt(localElements, indexes.start);
  const endElement = findVisibleElementAt(localElements, indexes.end - 1);

  const { startAnchor, endAnchor } = getAnchors(startElement, endElement);

  const formattingAttribute = getFormattingAttribute(e);

  if (!formattingAttribute) return;

  const attributes = { [formattingAttribute.key]: formattingAttribute.value };

  const exclusivePartner = EXCLUSIVE_ATTRIBUTES[formattingAttribute.key];
  if (exclusivePartner && formattingAttribute.value === "true") {
    attributes[exclusivePartner] = "false";
    editor.formatter.remove(exclusivePartner);
  }

  if (indexes.start === indexes.end) {
    Object.assign(pendingAttributes, attributes);
    return;
  }

  const formatting = {
    formattingId: { nodeId: myNodeId, counter: myCounter++ },
    start: startAnchor,
    end: endAnchor,
    attributes,
  };
  localFormattings.push(formatting);
  sendOrQueue({ type: "Formatting", data: formatting });
});
```

Слика 5.25. – Обрада команди за форматирање текста и креирање `CrdtFormatting` операције на страни клијента.

На слици 5.26. приказане су функције за реконструкцију опсега форматирања и одређивање активних атрибута за одређену позицију у тексту. Функција `resolveFormattingRanges` за сваки запис форматирања преводи почетно и крајње сидро у видљиве позиције текста помоћу функције `resolveAnchorToVisibleIndex`. Форматирања код којих није могуће одредити позицију или код којих је опсег празан се занемарују. За преостала форматирања формира се запис који садржи почетну и крајњу позицију, атрибуте и идентификатор форматирања.

Функција `activeAttributesAt` одређује атрибуте који су активни на задатој позицији. За сваки атрибут који припада опсегу проверава се да ли већ постоји изабрана вредност. Уколико се више форматирања преклапа за исти атрибут, као победничко се бира оно са већим `formattingId`, при чему се поређење врши помоћу функције `compareIds`. На овај начин се обезбеђује детерминистичко разрешавање преклапајућих форматирања, након чега се добијене вредности атрибута користе приликом приказа форматираног текста у едитору.

```
function resolveFormattingRanges() {
  const ranges = [];
  for (const formatting of localFormattings) {
    const start = resolveAnchorToVisibleIndex(formatting.start);
    const end = resolveAnchorToVisibleIndex(formatting.end);

    if (start === null || end === null || start >= end) continue;

    ranges.push({
      start,
      end,
      attributes: formatting.attributes,
      formattingId: formatting.formattingId,
    });
  }

  return ranges;
}

function activeAttributesAt(ranges, i) {
  const winners = {};

  for (const range of ranges) {
    if (i < range.start || i >= range.end) continue;

    for (const key of Object.keys(range.attributes)) {
      const current = winners[key];

      const isNewer =
        !current || compareIds(range.formattingId, current.formattingId) > 0;

      if (isNewer) {
        winners[key] = {
          formattingId: range.formattingId,
          value: range.attributes[key],
        };
      }
    }
  }

  const active = {};
  for (const key of Object.keys(winners)) {
    active[key] = winners[key].value;
  }
  return active;
}
```

Слика 5.26. – Функције `resolveFormattingRanges` и `activeAttributesAt` за одређивање опсега и активних атрибута форматирања.

На слици 5.27. приказана је функција `buildTagsFor`, која активне атрибуте форматирања претвара у одговарајуће HTML ознаке. За атрибуте који представљају CSS својства формирају се стилски делови који се смештају у `span` елемент. На основу мапе `STYLE_PROPERTY_FOR_ATTRIBUTE` одређује се одговарајуће CSS својство, док се његова вредност преузима из активног атрибута.

За атрибуте који се представљају HTML ознакама, као што су подебљан или курзиван текст, користи се мапа `TAG_FOR_ATTRIBUTE`. Ознаке се додају у листе за отварање и затварање, при чему се затварајуће ознаке додају обрнутим редоследом како би се обезбедила исправна структура угнежђених HTML елемената. На крају функција враћа објекат који садржи формиране отварајуће и затварајуће ознаке, које се користе приликом генерисања HTML садржаја документа.

```
function buildTagsFor(active) {
  const openTags = [];
  const closeTags = [];

  const styleParts = [];
  for (const key of Object.keys(active)) {
    if (key in STYLE_PROPERTY_FOR_ATTRIBUTE) {
      styleParts.push(`${STYLE_PROPERTY_FOR_ATTRIBUTE[key]}: ${active[key]}`);
    }
  }
  if (styleParts.length) {
    openTags.push(`<span style="${styleParts.join("; ")}">`);
    closeTags.unshift("</span>");
  }

  for (const key of Object.keys(active)) {
    if (key in TAG_FOR_ATTRIBUTE && active[key] === "true") {
      openTags.push(`<${TAG_FOR_ATTRIBUTE[key]}>`);
      closeTags.unshift(`</${TAG_FOR_ATTRIBUTE[key]}>`);
    }
  }

  return { openTags: openTags.join(""), closeTags: closeTags.join("") };
}
```

Слика 5.27. – Функција `buildTagsFor` за формирање HTML ознака на основу активних атрибута форматирања.

На слици 5.28. приказана је функција `renderFormattedHtml`, која на основу видљивих CRDT елемената и дефинисаних опсега форматирања генерише HTML садржај који се приказује у TinyMCE едитору. На почетку се позива функција `resolveFormattingRanges` како би се одредили видљиви опсежи форматирања.

Током проласка кроз видљиве елементе, знак за нови ред се обрађује као граница пасуса. Приликом промене активних атрибута форматирања, претходно отворене HTML ознаке се затварају, а помоћу функције `buildTagsFor` формирају се и отварају нове ознаке. На тај

начин се различити делови текста могу приказати са одговарајућим форматирањем, укључујући преклапајуће HTML елементе.

Пре додавања текста у резултујући HTML, вредност CRDT елемента се прослеђује функцији `escapeHtml`, чиме се спречава да специјални HTML карактери буду интерпретирани као део ознака. Након обраде свих елемената затварају се преостале отворене ознаке и пасус, а у случају празног документа генерише се празан пасус који TinyMCE може да прикаже.

```
function renderFormattedHtml(visibleElements) {
  const ranges = resolveFormattingRanges();

  let html = "";
  let paragraphOpen = false;
  let openActiveKey = null;
  let openCloseHtml = "";

  function closeOpenAttrs() {
    html += openCloseHtml;
    openActiveKey = null;
    openCloseHtml = "";
  }

  visibleElements.forEach((el, i) => {
    if (el.value === "\n") {
      closeOpenAttrs();
      html += paragraphOpen ? "</p>" : "<p><br></p>";
      paragraphOpen = false;
      return;
    }

    if (!paragraphOpen) {
      html += "<p>";
      paragraphOpen = true;
      openActiveKey = null;
      openCloseHtml = "";
    }

    const active = activeAttributesAt(ranges, i);
    const activeKey = JSON.stringify(active);

    if (activeKey !== openActiveKey) {
      closeOpenAttrs();
      const { openTags, closeTags } = buildTagsFor(active);
      html += openTags;
      openActiveKey = activeKey;
      openCloseHtml = closeTags;
    }

    html += escapeHtml(el.value);
  });

  if (paragraphOpen) {
    closeOpenAttrs();
    html += "</p>";
  }
  return html.length ? html : "<p><br></p>";
}
```

Слика 5.28. – Функција `renderFormattedHtml` за генерисање HTML приказа CRDT документа са примењеним форматирањем.

На слици 5.29. приказане су обраде догађаја `ElementsChanged` и `FormattingsChanged`, који се активирају приликом пријема ажурираног стања документа са сервера. У случају промене текстуалних елемената, локална реплика се ажурира кроз `localElements`, док се код промена форматирања ажурира `localFormattings`. Истовремено се помоћу функције `buildLamportCounter` одређује следећа вредност локалног бројача на основу идентификатора примљених елемената, односно форматирања.

Након ажурирања локалног стања, постављањем `isApplyingRemoteChange` на `true` означава се да промена потиче са сервера. Затим се позива `renderHtml`, чиме се на основу новог стања поново генерише HTML садржај и поставља у TinyMCE едитор. Овај механизам спречава да се примљене удаљене измене поново обраде као локалне измене корисника.

```
connection.on("ElementsChanged", (elements) => {
  myCounter = buildLamportCounter(elements.map((e) => e.crdId));
  localElements = elements;
  isApplyingRemoteChange = true;
  editor.setContent(renderHtml());
  isApplyingRemoteChange = false;
});

connection.on("FormattingsChanged", (formattings) => {
  myCounter = buildLamportCounter(formattings.map((f) => f.formattingId));
  localFormattings = formattings;
  isApplyingRemoteChange = true;
  editor.setContent(renderHtml());
  isApplyingRemoteChange = false;
});
```

Слика 5.29. – Обрада промена примљених са сервера путем SignalR-а и ажурирање садржаја TinyMCE едитора.

На слици 5.30. приказан је механизам за чување CRDT операција насталих током рада у офлајн режиму. Операције се смештају у локално складиште прегледача помоћу `localStorage`, при чему се за сваки документ користи засебан кључ. Функције `getOfflineQueue` и `saveOfflineQueue` омогућавају читање и ажурирање локалног реда, док `queueOfflineOperation` додаје нову операцију у ред.

Функција `sendOrQueue` проверава стање SignalR везе. Уколико је веза доступна, операција се одмах прослеђује одговарајућој методи на серверу. У супротном, или у случају грешке приликом слања, операција се смешта у офлајн ред како би се сачувала за каснију синхронизацију.

Након поновног успостављања везе позива се `flushOfflineQueue`, која преузима све сачуване операције и прослеђује их серверу путем методе `ApplyOfflineOperations`. Након успешног слања ред се празни, чиме се омогућава да се локално настале измене накнадно примене на серверу без губитка података.

```

async function flushOfflineQueue() {
  const queue = getOfflineQueue();
  if (queue.length === 0) return;

  await connection.invoke("ApplyOfflineOperations", queue, docId);

  saveOfflineQueue([]);
}

function getOfflineQueue() {
  const stored = localStorage.getItem(OFFLINE_QUEUE_KEY);

  if (!stored) {
    return [];
  }
  return JSON.parse(stored);
}

function saveOfflineQueue(queue) {
  localStorage.setItem(OFFLINE_QUEUE_KEY, JSON.stringify(queue));
}

function queueOfflineOperation(op) {
  const queue = getOfflineQueue();
  queue.push(op);
  saveOfflineQueue(queue);
}

async function sendOrQueue(operation) {
  if (connection.state === signalR.HubConnectionState.Connected) {
    try {
      await connection.invoke(
        HUB_METHOD_FOR_TYPE[operation.type],
        operation.data,
        docId,
      );
    } catch (error) {
      console.error("[crdtsync] failed to send operation", error);

      queueOfflineOperation(operation);
    }

    return;
  }

  queueOfflineOperation(operation);
}

```

Слика 5.30. – Управљање редом операција у офлајн режиму и њихова накнадна синхронизација са сервером.

На слици 5.31. приказана је регистрација додатка у оквиру TinyMCE библиотеке, као и функција за генерисање јединственог идентификатора клијента који се повезује са сервером. Функција `getOrCreateNodeId` преузима идентификатор клијента из `localStorage`-а. Уколико идентификатор не постоји, генерише се нова случајна вредност, која се затим чува за будуће сесије.

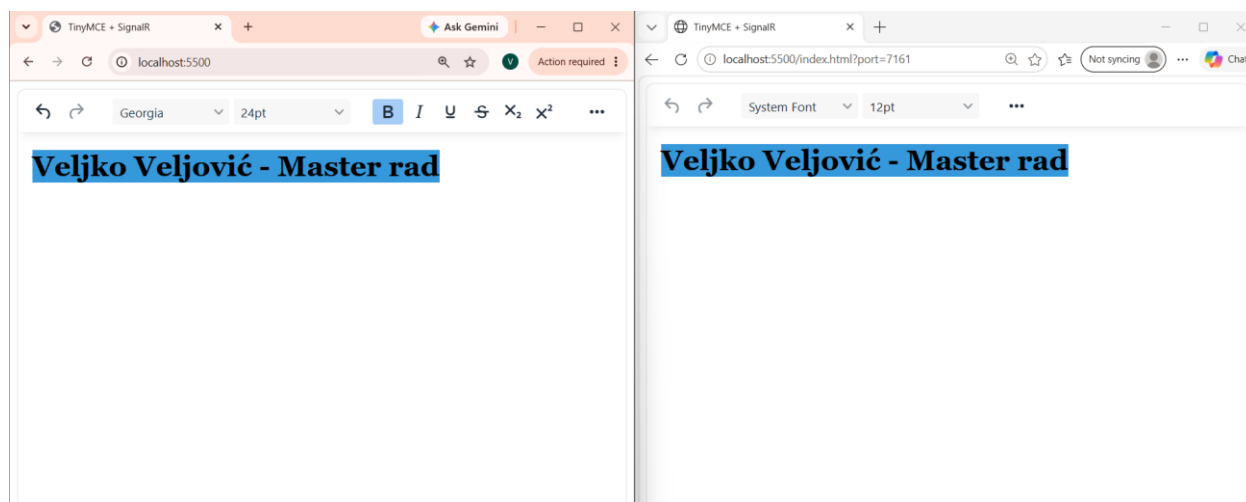
```

1  tinymce.PluginManager.add("crdtsync", function (editor) {
2      editor.options.register("crdtsync_hub_url", {
3          processor: "string",
4          default: "default",
5      });
6      editor.options.register("crdtsync_doc_id", {
7          processor: "string",
8          default: "default",
9      });
10     const hubUrl = editor.options.get("crdtsync_hub_url");
11     const docId = editor.options.get("crdtsync_doc_id");
12
13     function getOrCreateNodeId() {
14         let id = localStorage.getItem("crdtsync_node_id");
15         if (!id) {
16             id = Math.floor(Math.random() * 1000000);
17             localStorage.setItem("crdtsync_node_id", id);
18         }
19
20         return parseInt(id, 10);
21     }

```

Слика 5.31. – Приказ регистрације crdtsync додатка у TinyMCE библиотеци и функције getOrCreateNodeId за генерисање и чување јединственог идентификатора клијента.

На слици 5.32. приказан је практичан рад реализоване апликације са два TinyMCE едитора покренута на различитим порттовима и повезана са различитим серверским инстанцама. Уношењем текста у један едитор измена се путем серверских инстанци прослеђује другом едитору, где се приказује ажурирано стање документа. На овај начин демонстрирана је синхронизација измена између два клијента и практична примена реализованог CRDT механизма.



Слика 5.32. – Приказ рада апликације са два TinyMCE едитора повезана са различитим серверским инстанцама.

6. Закључак

Колаборативно уређивање текста у реалном времену представља значајан изазов у развоју дистрибуираних система, пре свега због потребе за очувањем конзистентности документа приликом истовремених измена више корисника. У оквиру овог рада анализирана су два основна приступа решавању овог проблема, Operational Transformation и Conflict-free Replicated Data Type, при чему су размотрене њихове основне карактеристике, предности и ограничења. Посебна пажња посвећена је CRDT приступу и његовим особинама које омогућавају да више реплика документа конвергира ка истом стању без потребе за централним механизмом који би трансформисао и усклађивао конкурентне операције.

Кроз анализу модела репликације у дистрибуираним системима, као и state-based и operation-based CRDT приступа, успостављена је теоријска основа за разумевање начина на који се текст може представити у облику дистрибуиране структуре података. Размотрени су принципи конвергенције, коначне конзистентности, комутативности, асоцијативности и идемпотентности, као и проблем испреплетаности елемената приликом конкурентног уметања текста. У том контексту анализиран је Fugue алгоритам као решење проблема испреплетаности, док је у имплементираним CRDT механизму редослед конкурентно уметнутих елемената одређен детерминистичким поређењем њихових идентификатора. За подршку богатом форматирању текста размотрен је и Peritext модел, који омогућава представљање форматирања независно од тренутних позиција карактера у документу.

На основу спроведене теоријске анализе, у практичном делу рада реализован је open-source систем за колаборативно уређивање текста интегрисан са TinyMCE едитором. Основу система чини CrdtCore библиотека, која садржи CRDT логику независну од комуникационог и складишног слоја. Управљање комуникацијом са клијентима реализовано је у оквиру CrdtServer апликације коришћењем SignalR-a, док се синхронизација CRDT операција између различитих серверских инстанци остварује путем RabbitMQ-a.

На клијентској страни развијен је crdtsync TinyMCE додатак који локалне измене преводи у CRDT операције уметања и брисања, док се форматирање текста представља помоћу CRDT структура заснованих на сидрима. На овај начин форматирање остаје везано за логичке позиције у CRDT структури, а не за тренутне индексе карактера, што омогућава његово правилно разрешавање и након конкурентних измена документа. Реализован је и механизам за реконструкцију форматираног HTML садржаја који се приказује у TinyMCE едитору.

Поред основне синхронизације, систем подржава рад у офлајн режиму. Операције настале током прекида комуникације привремено се чувају у локалном складишту клијента, а након поновног успостављања везе прослеђују се серверу и укључују у процес синхронизације. На тај начин корисник може да настави са уређивањем документа и током привремене недоступности сервера, без губитка локално насталих измена.

Реализовано решење показује да је применом CRDT приступа могуће изградити функционалан систем за колаборативно уређивање текста који подржава конкурентне измене, синхронизацију више реплика, богато форматирање текста и рад у офлајн режиму. Комбинација CrdtCore библиотеке, CrdtServer апликације, SignalR-a, RabbitMQ-a и TinyMCE додатка представља основу за даљи развој система и његову примену у различитим сценаријима који захтевају дистрибуирано колаборативно уређивање.

7. Литература

1. M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, O'Reilly Media, 2017.
2. M. Shapiro, N. Preguiça, C. Baquero and M. Zawirski, "A Comprehensive Study of Convergent and Commutative Replicated Data Types," INRIA Research Report, 2011. <https://inria.hal.science/inria-00555588>
3. M. Shapiro, N. Preguiça, C. Baquero and M. Zawirski, "Conflict-Free Replicated Data Types," Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems, 2011.
4. N. Preguiça, C. Baquero and M. Shapiro, "Conflict-Free Replicated Data Types (CRDTs)," *Encyclopedia of Big Data Technologies*, Springer, 2019. <https://arxiv.org/abs/1805.06358>
5. "Conflict-Free Replicated Data Types (CRDTs)," crdt.tech. <https://crdt.tech/>
6. M. Kleppmann and D. Gomes, "Interleaving Anomalies in Collaborative Text Editors," Proceedings of the 6th Workshop on Principles and Practice of Consistency for Distributed Data, 2019. <https://martin.kleppmann.com/papers/interleaving-papoc19.pdf>
7. M. Weidner, D. Gomes, M. Kleppmann and A. Beresford, "The Art of the Fugue: Minimizing Interleaving in Collaborative Text Editing," 2023. <https://arxiv.org/abs/2305.00583>
8. I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force, 2011. <https://www.rfc-editor.org/rfc/rfc6455>
9. "TinyMCE Documentation." <https://www.tiny.cloud/docs/tinymce/latest/>
10. "Introduction to SignalR," Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/signalr/overview/getting-started/introduction-to-signalr>
11. G. Litt, S. Lim, M. Kleppmann and P. van Hardenberg, "Peritext: A CRDT for Collaborative Rich Text Editing," *Proceedings of the ACM on Human-Computer Interaction*, 2022. <https://www.inkandswitch.com/peritext/cscw-publication.pdf>
12. H.-G. Roh, M. Jeon, J.-S. Kim and J. Lee, "Replicated Abstract Data Types: Building Blocks for Collaborative Applications," *Journal of Parallel and Distributed Computing*, 2011.
13. M. Kleppmann and A. R. Beresford, "A Conflict-Free Replicated JSON Datatype," *IEEE Transactions on Parallel and Distributed Systems*, 2017. <https://arxiv.org/abs/1608.03960>
14. C. A. Ellis and S. J. Gibbs, "Concurrency Control in Groupware Systems," Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data, 1989.
15. "AMQP 0-9-1 Model Explained," RabbitMQ Documentation. <https://www.rabbitmq.com/tutorials/amqp-concepts>
16. "Overview of ASP.NET Core SignalR," Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction>
17. M. Kleppmann, A. Wiggins, P. van Hardenberg and M. McGranaghan, "Local-first Software: You Own Your Data, in Spite of the Cloud," Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, 2019. <https://martin.kleppmann.com/2019/10/23/local-first-at-onward.html>
18. C. Sun and C. Ellis, "Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements," Proceedings of the 1998 ACM Conference on Computer Supported Cooperative Work, 1998. <https://doi.org/10.1145/289444.289469>